

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY



FINAL PROJECT REPORT

BEVIETNAM: AN AGENT-BASED SMART TOURISM SYSTEM FOR VIETNAM

Course ID: CSC10014
Course Name: Computational Thinking
Semester: II, 2026
Class ID: 24CTT6
Group ID: Group 09
Instructors & TAs: MSc. Thanh Tuan Ho & MSc. Tuan Anh Mai

Student ID and Name:

24120068 Nguyễn Phúc Khang (Leader)
24120103 Hoàng Trọng Nghĩa
24120441 Nguyễn Việt Thái
24120453 Trần Hồng Thiên
24120407 Nguyễn Dương Hoàng Phi
24120273 Nguyễn Thành Công

Table of Contents

1	Ý Tưởng Đề Tài	1
1.1	Bối Cảnh Thực Tiễn và Bài toán	1
1.2	Problem Statement	2
1.3	Tầm Nhìn Sản Phẩm	3
1.4	Đối Tượng Khách Hàng Mục Tiêu	3
2	Phân Tích Bài Toán	3
2.1	Tổng Quan Cách Tiếp Cận	3
2.2	5W1H và 5 Whys	4
2.3	Decomposition	5
2.4	Interact Diagram	9
2.5	Data Model	10
3	Tổng Quan Hệ Thống	11
3.1	Main Actors	12
3.2	Core Features	13
3.3	Explanation of Core Features	13
3.3.1	Curated Place Discovery	14
3.3.2	Dynamic Recommendation Feed	14
3.3.3	Realtime Bubble Map	15
3.3.4	Storyline Mission Path	15
3.3.5	Vision Capture Verification	16
3.3.6	Bilingual Experience	17
3.4	Pipeline Tổng Thể Hệ Thống	17
3.5	Innovation Highlights	18
4	Pattern Recognition	19
5	Abstraction	20
6	System and Algorithm Design	22
6.1	Algorithm Design	22
6.2	Ranking Algorithm	23
6.2.1	Personally Ranking Feed	24
6.2.2	Grading for Mission Path	25
6.2.3	Dynamic Bubble Marker	26
6.3	Thuật Toán Điều Hướng và Xác Minh	26
6.4	Evaluation	27
7	Triển khai Hệ thống	28
7.1	Kiến Trúc Tổng Thể	28
7.2	Kiến Trúc theo Mô Hình C4	29
7.2.1	Abstract Layer	29



7.2.2	Frontend Layer	30
7.2.3	Backend Layer	30
7.2.4	AI Service	31
7.3	Lớp Dữ Liệu và API	31
7.4	Realtime Bubble Map	31
7.5	Storyline Mission Path	32
7.6	Vision Capture Verification	32
7.7	Mô Hình AI Tự Host	34
8	Kiểm Thử và Đánh Giá	34
8.1	Phương Pháp Kiểm Thử	35
8.2	Kết Quả Kiểm Thử Chính	35
8.3	Đánh Giá theo Bốn Tiêu Chí	35
9	Triển Khai	36
9.1	Bố Trí Triển Khai	36
9.2	Tự Động Hóa Khởi Tạo	37
9.3	Bảo Mật và Cấu Hình Bí Mật	37
10	Kết Luận và Hướng Phát Triển	38
10.1	Kết Luận	38
10.2	Hướng Phát Triển	38

1 Ý Tưởng Đề Tài

1.1 Bối Cảnh Thực Tiễn và Bài toán

Trong những năm trở lại đây, du lịch Việt Nam đã trở thành một đầu tàu mũi nhọn đóng vai trò rất lớn đối với nền kinh tế Việt Nam. Nó không chỉ sở hữu tài nguyên phong phú và đa dạng, mà còn có rất nhiều các di tích lịch sử ngàn năm văn hiến, danh lam thắng cảnh kỳ vĩ đến các di sản văn hóa phi vật thể và ẩm thực đặc sắc của từng vùng miền. Trong những năm gần đây, đặc biệt là giai đoạn phục hồi mạnh mẽ sau đại dịch, ngành du lịch đã ghi nhận những bước chuyển mình vượt bậc với các số liệu tăng trưởng ấn tượng. Theo số liệu chính thức từ Tổng cục Thống kê, vào năm 2025, Việt Nam đã đón gần 21.2 triệu lượt khách quốc tế, đạt mức tăng trưởng hơn 20% so với năm 2024 và thiết lập kỷ lục lịch sử mới về lượng khách ngoại quốc ghé thăm trong một năm [1]. Bên cạnh đó, thị trường du lịch nội địa cũng ghi nhận sức phát triển sôi động khi phục vụ khoảng 135.5 triệu lượt khách, từ đó nâng tổng thu từ khách du lịch lần đầu tiên trong lịch sử vượt mốc 1 triệu tỷ đồng [2]. Nhằm tiếp tục phát huy đà tăng trưởng này, ngành du lịch nước nhà đã đặt mục tiêu lớn cho năm 2026 với việc đón 25 triệu lượt khách quốc tế và đạt tổng doanh thu ước tính khoảng 1.125 triệu tỷ đồng [3], khẳng định vai trò cốt lõi của du lịch như một ngành kinh tế mũi nhọn của đất nước. Tuy nhiên, dù cho với những số liệu tích cực đó, ngành du lịch nước vẫn còn nhiều hạn chế dẫn đến nhiều bất cập cho du khách trong nước và ngoài nước khi đến trải nghiệm.

Trước hết là nước ta đang đối mặt với một thực trạng là tỷ lệ khách quay lại khá thấp. Khi chúng ta so sánh với các quốc gia trong khu vực như Thái Lan — nơi có tỷ lệ khách quay lại đạt đến gần 80% thì tại Việt Nam, tỷ lệ này chỉ dao động từ 10% đến 40% [4]. Điều này phản ánh thực tế rằng các sản phẩm du lịch hiện nay đang thiếu đi một cái gì đó để níu kéo khách hàng, khiến du khách thường chỉ tới một lần duy nhất chỉ để ngắm cảnh mà không thấy được lý do mạnh mẽ để quay lại thêm một lần nữa. Đặc biệt quan trọng là công tác quản lý điểm đến tại nhiều địa phương bộc lộ ra nhiều hạn chế, chưa loại bỏ được tình trạng đeo bám, ép giá, hay vấn đề vệ sinh môi trường, qua đó để lại những ấn tượng xấu trong lòng bạn bè quốc tế.

Tiếp đó chính là việc thông tin và dữ liệu phân tán khắp mọi nơi trong thời đại số. Hiện nay, dữ liệu về các điểm đến, hoạt động văn hóa nghệ thuật, hay ẩm thực đặc sản phân tán ra trên nhiều nền tảng số như Google Maps, các trang mạng xã hội, các blog cá nhân hay các ứng dụng cục bộ của từng địa phương. Điều này buộc du khách phải tiến hành công việc tìm kiếm và đối chiếu thông tin khá thủ công trước và trong suốt chuyến đi. Mặt khác, công cụ số hóa hiện nay hầu như chỉ cung cấp thông tin mặt ngoài như tọa độ, thời gian đóng mở cửa và đánh giá sao chung chung, hoàn toàn thiếu đi khả năng giải nghĩa các câu chuyện lịch sử, giá trị nhân văn và ý niệm văn hóa sâu xa đằng sau các di tích cổ kính.

Và cuối cùng là việc thiếu hụt các giải pháp công nghệ tự động hóa tùy theo điều kiện thực tế. Trải nghiệm của du khách tại điểm đến bị chi phối khá nhiều bởi các yếu tố ngoại cảnh thay đổi liên tục như thời tiết, tình trạng ùn tắc giao thông, khoảng cách địa lý hay mật độ đông đúc. Một di sản lịch sử có giá trị văn hóa lớn, nhưng nếu du khách tới tham quan vào thời điểm mưa bão hay quá tải chen chúc thì trải nghiệm họ thu nhận được sẽ kém hiệu quả đi nhiều. Rõ ràng, việc các hệ thống hiện nay không thể tự động điều chỉnh đề xuất tùy theo bối cảnh môi trường thời gian thực đã tạo ra khoảng cách rất lớn giữa kỳ vọng của du khách và khả năng đáp ứng của dịch vụ.

Từ những khó khăn thực tế nêu trên, ta đặt ra câu hỏi rằng làm sao ta có thể phát triển một giải pháp công nghệ có khả năng tổng hợp các nguồn dữ liệu phân tán thành một hệ thống chung, đồng thời tự động hóa việc đưa ra quyết định hỗ trợ du khách tùy theo điều kiện ngoại cảnh thay đổi liên tục. Điều

này tạo ra nhu cầu cần thiết cho một mô hình hệ thống có khả năng thích ứng bối cảnh, xử lý dữ liệu lớn về văn hóa bản địa và dẫn dắt hành trình du lịch của người dùng một cách hiệu quả.

1.2 Problem Statement

Trước hết nếu ta nhìn theo góc độ của người dùng, đây là một hệ thống thông minh hỗ trợ khách du lịch tìm kiếm địa điểm du lịch thú vị, nhưng song với đó cũng cung cấp thêm nhiều kiến thức về văn hóa của nơi đó nhằm tạo nên một sự hứng thú cho họ. Còn nếu nhìn từ góc độ kỹ thuật, thì lại là một bài toán tối ưu hóa quyết định với nhiều mục tiêu và ràng buộc thay đổi theo thời gian thực, đòi hỏi ta phải mô hình hóa được sở thích của người dùng từ thực thể đời thường, thành một đối tượng toán học và từ đó có thể cá nhân hóa theo sở thích, tương tác của họ. Đây chính là động lực chính để phát triển hệ thống **BeVietnam**, một giải pháp du lịch thông minh dựa trên mô hình **agent-based system** nhằm giảm tình trạng phân tán thông tin, hỗ trợ đề xuất linh hoạt tùy theo điều kiện thực tế và dẫn dắt hành trình trải nghiệm thông qua các câu chuyện. Và để làm được điều đó, nhóm đã đề ra ba tính năng chính và là mục tiêu cốt lõi của dự án như sau:

Đầu tiên, phần lớn các du khách khi đến một điểm du lịch nào đó thì học chủ yếu để ngắm cảnh và vui chơi. Tuy nhiên, nếu một địa điểm đó chỉ được dùng để ngắm xong rồi về thì nó sẽ tạo cho họ một cảm giác chán và thiếu hụt đi cái gì đó. Họ chỉ ngắm nhưng họ không hiểu thật sự phía sâu bên trong mỗi cảnh đẹp, mỗi kỳ quan kia đang chứa đựng điều gì ở đó. Chính vì lẽ đó để giảm bớt vấn đề cảnh quan, địa điểm thiếu đi chiều sâu dẫn đến tỷ lệ quay lại thấp, nhóm đã nghiên cứu và xây dựng tính năng nhiệm vụ văn hóa storyline theo phong cách geocaching thực địa. Tại nơi đây, người dùng sẽ đóng vai trò như là một **nhà lữ hành** thực thụ, họ sẽ được giao cho những nhiệm vụ xoay quanh các kiến trúc cổ, các danh lam thắng cảnh, các địa vui chơi mới và sau đó họ sẽ tiến hành chụp ảnh, checkin nộp lên hệ thống để kiểm chứng và hoàn thành nhiệm vụ. Hệ thống tổ chức toàn bộ nhiệm vụ thành một **Mission Path** trực quan để giữ chân người dùng trong đó mỗi nút trên lộ trình là một nhiệm vụ văn hóa được nạp trực tiếp từ kho câu hỏi đã được xây dựng từ trước từ rất nhiều cuốn sách văn hóa về điểm đó. Thay vì chỉ ngắm cảnh thụ động, du khách buộc phải di chuyển đến tọa độ GPS của điểm đến và chụp ảnh minh chứng các yếu tố lịch sử, kiến trúc. Ảnh minh chứng này được một mô hình thị giác chấm điểm theo một thang đánh giá để quyết định mở khóa nút tiếp theo, qua đó biến chuyến đi thành hành trình tương tác rõ ràng hơn, ly kỳ hơn và lý thú hơn.

Tiếp theo, nhằm triệt tiêu vấn đề phân tán thông tin du lịch, nhóm phát triển tính năng bảng tin gợi ý feed thông minh. Tính năng này hoạt động giống như một bộ lọc trung tâm, tự động tổng hợp nguồn tri thức văn hóa chính thống từ nhiều cuốn sách và trang web khác nhau để trả về các bài viết giới thiệu điểm đến trực quan mới mẻ và thú vị, giúp du khách nắm bắt trọn vẹn giá trị di sản mà không phải tìm kiếm thủ công trên nhiều nền tảng khác nhau. Khi phát triển tính năng này, nhóm làm ra với mong muốn là trước khi họ đến một nơi nào đó, họ sẽ có thể tìm hiểu trước về địa điểm đó, biết được địa điểm sắp tới của mình là gì, biết được tại sao nó lại nhiều người đến thăm, biết tại sao nó nổi tiếng, từ đó mà có thể tọa nên một sự tò mò và kích thích từ sâu bên trong họ.

Cuối cùng, để khắc phục điểm yếu về việc phản ứng theo điều kiện ngoại cảnh thực tế, nhóm thiết kế tính năng bubble map tương tác hiển thị các POI thời gian thực quanh vị trí người dùng. Hệ thống lấy vị trí GPS thực của du khách, truy vấn các địa điểm trong bán kính 3km qua nguồn dữ liệu **Foursquare**, đồng thời thu thập thời tiết khu vực hiện tại (nhiệt độ, chỉ số UV ước lượng, lượng mưa) bằng **OpenWeather API**. Kích thước mỗi bong bóng co giãn theo khoảng cách thực tế từ địa điểm tới người dùng, qua đó

giúp du khách nhận biết nhanh nơi nào đáng cân nhắc nhất tại thời điểm hiện tại. Điều này làm ra bởi lẽ, khi du khách muốn đi một nơi nào đó họ sẽ rất muốn biết rằng nơi đó thời tiết như nào có đang mưa hay không, có đông hay không, có nhiều review đánh giá tốt hay không? Và đặc biệt là làm sao để họ có thể tiếp cận được những điều đó. Ví dụ như họ vô Google Maps thì làm thế nào mà họ có thể biết được quán này muốn đến, quán này nên đi, điểm kia nên tới một cách tiết kiệm nhất có thể, họ không thể nào mà dành hàng giờ đồng hồ để kiểm tra, tìm hiểu về nó xong rồi khi sắp tới nơi thì lại mắc mưa.

1.3 Tầm Nhìn Sản Phẩm

BeVietnam được định hướng phát triển thành một hệ sinh thái du lịch văn hóa thông minh, đầu tiên sẽ tập trung thí điểm tại Huế. Thực ra ở giai đoạn đầu nhóm từng muốn bao phủ nhiều tỉnh thành cùng lúc, nhưng khi bắt tay chuẩn bị dữ liệu văn hóa thì thấy nội dung quá dàn trải và khó kiểm chứng nguồn, nên nhóm chủ động thu hẹp lại còn điểm duy nhất là Huế để giữ được chất lượng dữ liệu và nội dung. Bởi lẽ khi thực hiện điều này, nhóm phải bỏ thời gian công sức rất nhiều để tìm kiếm các nguồn tài liệu chính thống, các cuốn sách hay về điểm đó, sau đó sẽ tiến hành quá trình ingest và tạo câu hỏi, đòi hỏi một khoảng thời gian dài và nhiều công sức. Chính vì lẽ đó trong khuôn khổ đề án này nhóm chỉ tập trung vào tỉnh thành đó chính là Huế.

Sản phẩm hướng đến các câu hỏi cho khách du lịch là *nên đi đâu vào lúc này, vì sao địa điểm đó đáng đi và khi đến đó thì nên làm gì* thông qua đó mà hệ thống có thể giúp họ có câu trả lời cho riêng mình. Từ tầm nhìn cốt lõi đó, sản phẩm được triển khai với ba trải nghiệm tương tác chính như sau: **Một là, bảng tin gợi ý địa điểm tương tự một feed mạng xã hội năng động**, trong đó mỗi đề xuất được tính toán và sắp xếp theo sự kết hợp giữa vị trí của người dùng, thời tiết hiện tại, điều kiện giao thông thực tế và sở thích cá nhân. **Hai là, bubble map hiển thị trực quan các điểm đến** với kích thước bong bóng thay đổi theo mức độ phù hợp của từng địa điểm ở thời điểm hiện tại. **Ba là, hệ thống nhiệm vụ văn hóa** theo phong cách geocaching, buộc người dùng phải trực tiếp di chuyển đến thực địa, check-in và chụp ảnh minh chứng để mở khóa các nội dung giải nghĩa văn hóa sâu sắc tiếp theo.

1.4 Đối Tượng Khách Hàng Mục Tiêu

Đối tượng người dùng mục tiêu của hệ thống *BeVietnam* bao gồm nhiều loại khách hàng khác nhau từ trong nước tới ngoài nước. Trước hết, nhóm du khách nội địa và quốc tế có nhu cầu tìm hiểu sâu sắc về lịch sử, kiến trúc cổ kính và ẩm thực độc đáo tại địa phương nhưng thường gặp rào cản về mặt ngôn ngữ hoặc thiếu thông tin giải nghĩa văn hóa có hệ thống. Kế đến, nhóm người đam mê du lịch tự túc và mong muốn tự chủ hoàn toàn lịch trình của mình, rất cần các đề xuất cá nhân hóa linh hoạt theo thời gian thực thay vì tuân theo các tour tuyến cố định và nhàm chán. Sau cùng, nhóm người dùng trẻ yêu thích khám phá công nghệ mới và các trải nghiệm trò chơi hóa, mong muốn tiếp thu kiến thức văn hóa một cách tự nhiên, chủ động thông qua việc hoàn thành các thử thách thực địa đầy thú vị.

2 Phân Tích Bài Toán

2.1 Tổng Quan Cách Tiếp Cận

Để giải quyết được bài toán cũng như là triển khai được sản phẩm thì nó đòi hỏi ta phải có kỹ năng phân tích vấn đề, ta cần biết được ta sẽ cần những gì và sẽ làm những gì chứ không thể nào bắt tay vào

làm liên. Chính vì lẽ đó để thực hiện được điều đó, nhóm đã áp dụng các kỹ năng và kiến thức của bộ môn Tư duy tính toán để có thể tận dụng các bộ khuôn tư duy trong phát triển phần mềm. Và rõ ràng bài toán được ra không chỉ là viết ra một ứng dụng có giao diện đẹp hay kết nối được với một vài API, mà sâu xa hơn, ta phải tìm được một cách *mô hình hóa* vấn đề sao cho máy tính có thể xử lý được một cách rõ ràng, có kiểm soát và có khả năng mở rộng về sau. Đây chính là lý do vì sao nhóm lựa chọn cách tiếp cận **Computational Thinking** làm nền tảng để phân tích và phát triển hệ thống này [5, 6, 7].

Tư duy tính toán không chỉ là việc lập trình bằng một ngôn ngữ cụ thể, mà nó là một bộ khuôn tư duy giúp ta phân rã một vấn đề phức tạp ngoài thực tế thành các thành phần nhỏ hơn, nhận diện quy luật lặp lại, trừu tượng hóa những yếu tố cốt lõi và cuối cùng xây dựng ra một quy trình giải có thể thực thi được bằng máy tính [6]. Khi áp dụng vào BeVietnam, ta có thể thấy rằng đây là một ví dụ rất rõ ràng của một bài toán thực tế có độ phức tạp cao, bởi nó đồng thời liên quan đến dữ liệu du lịch, ngữ cảnh thời gian thực, hành vi người dùng, nội dung văn hóa và khả năng ra quyết định tự động. Và như vậy sau những phân tích về bối cảnh của ngành du lịch ở trên nhóm cũng đã tìm các pain point chính hiện thời mà khách hàng đang còn mắc phải.

Table 1: Các Pain Point chính dẫn tới bài toán BeVietnam

Pain Point	Biểu hiện trong trải nghiệm du lịch	Hệ quả đối với hệ thống cần xây dựng
Thông tin phân tán	Người dùng phải chuyển qua nhiều nền tảng như bản đồ, blog, review và mạng xã hội để ghép thông tin về một địa điểm.	Hệ thống phải có một lớp dữ liệu điểm đến đủ thống nhất để giảm công việc tìm kiếm và đối chiếu thủ công.
Thiếu chiều sâu văn hóa	Người dùng biết nơi nào nổi tiếng nhưng không hiểu vì sao nơi đó quan trọng về mặt lịch sử hoặc văn hóa.	Hệ thống không được dừng ở việc liệt kê POI, mà phải trả thêm lớp giải nghĩa văn hóa có thể đọc và hiểu được.
Không biết nên đi đâu lúc này	Một địa điểm có thể rất hay nhưng lại không phù hợp ở thời điểm hiện tại vì mưa, xa, kẹt xe hoặc quá đông.	Hệ thống phải đưa yếu tố ngữ cảnh thời gian thực vào bài toán xếp hạng thay vì chỉ dùng danh sách tĩnh.
Không biết nên làm gì tiếp theo	Sau khi chọn được địa điểm, người dùng vẫn dễ rơi vào trạng thái tham quan thụ động, không có hành động cụ thể để khám phá sâu hơn.	Hệ thống cần có cơ chế storyline hoặc quest chain để biến việc đọc thông tin thành hành trình có hướng dẫn.
Khó xác minh trải nghiệm thực địa	Nếu không có cơ chế kiểm chứng, việc hoàn thành nhiệm vụ chỉ còn là thao tác trên giao diện chứ không còn gắn với địa điểm thật.	Hệ thống phải có lớp GPS + capture verification để nối trạng thái số với hành vi ngoài đời.

2.2 5W1H và 5 Whys

Trong giai đoạn đầu, khi những thông tin phân tích chỉ mới dừng lại ở mức độ là ý tưởng thì việc triển khai vẫn chưa được thực hiện. Do đó ta cần thêm một giai đoạn để có thể làm rõ lên những yêu cầu và

mong muốn đặt ra khi làm sản phẩm này nhóm dùng bộ câu hỏi **5W1H** để khóa phạm vi bài toán và dùng **5 Whys** để truy tìm nguyên nhân gốc. Trước hết, 5W1H giúp nhóm trả lời sáu câu hỏi nền tảng *Cái gì, Tại sao, Ai, Ở đâu, Khi nào, Như thế nào* nhằm chốt đúng phạm vi của bài toán, tổng hợp trong Bảng 2 [8].

Table 2: Phân tích 5W1H để khóa phạm vi bài toán BeVietnam

Câu hỏi	Câu trả lời cho BeVietnam
What — Cái gì	Giúp du khách chọn một địa điểm có ý nghĩa văn hóa và phù hợp để đi <i>ngay tại thời điểm hiện tại</i> .
Why — Tại sao	Vì sản phẩm du lịch thiếu chiều sâu, thông tin phân tán và không phản ứng theo bối cảnh, khiến du khách khó quyết định và ít quay lại.
Who — Ai	Du khách nội địa và quốc tế là người dùng cuối; nhóm phát triển đóng vai trò vận hành và kiểm thử pilot.
Where — Ở đâu	Khóa phạm vi pilot tại Huế để kiểm soát chất lượng dữ liệu và nội dung văn hóa.
When — Khi nào	Quyết định theo bối cảnh thời gian thực (vị trí, thời tiết, thời điểm) thay vì tra cứu tĩnh.
How — Như thế nào	Bằng suitability score giải thích được, storyline tương tác, và xác minh thực địa qua GPS kết hợp capture.

Sau 5W1H, nhóm tiếp tục dùng **5 Whys** để đào sâu nguyên nhân gốc của các Pain Point chính. Ví dụ với hiện tượng “du khách không biết nên đi đâu bây giờ”, nếu chỉ dừng ở bề mặt thì ta rất dễ kết luận rằng “cần nhiều dữ liệu hơn”. Nhưng khi hỏi tiếp nhiều lần, nhóm rút ra được chuỗi nguyên nhân như sau.

Table 3: Phân tích 5 Whys cho bài toán “du khách không biết nên đi đâu bây giờ”

Bước	Câu hỏi Why	Kết quả phân tích
Why 1	Tại sao du khách không biết nên đi đâu?	Vì có quá nhiều lựa chọn rời rạc giữa Google Maps, blog, review và mạng xã hội.
Why 2	Tại sao nhiều lựa chọn lại gây bối rối?	Vì các nền tảng đó không xếp hạng theo bối cảnh thực tế của chuyến đi.
Why 3	Tại sao không xếp hạng được theo bối cảnh?	Vì dữ liệu thời tiết, giao thông, khoảng cách và mức độ phù hợp chưa được gom về cùng một mô hình quyết định.
Why 4	Tại sao chưa có mô hình quyết định chung?	Vì feed, map và place detail thường được phát triển như các bề mặt hiển thị tách rời chứ không cùng dùng một lõi scoring.
Why 5	Tại sao điều đó nguy hiểm?	Vì người dùng nhìn thấy thông tin nhưng không nhận được câu trả lời rõ ràng cho hành động kế tiếp.

Điều này dẫn trực tiếp tới một quyết định quan trọng trong thiết kế BeVietnam: **feed và bubble map phải dùng cùng một suitability score, và score đó phải giải thích được** [8]. Đây không còn là một ý tưởng chung chung, mà là kết quả cụ thể của bước phân tích bài toán bằng 5 Whys.

2.3 Decomposition

Trong thực tế, khi ta đối mặt với một bài toán lớn, một yêu cầu của khách hàng, ta không trực tiếp suy luận là làm sao để ta làm được điều đó, mà ta sẽ phải chia bài toán tổng thể thành các bài toán con

nhỏ hơn và có thể kiểm soát được. Ý tưởng cốt lõi của **decomposition** là thay vì cố gắng giải trực tiếp toàn bộ vấn đề “xây dựng một hệ thống du lịch thông minh cho Việt Nam”, ta sẽ phân rã nó thành các thành phần nhỏ hơn, mỗi thành phần có đầu vào, đầu ra và mục tiêu riêng biệt [6, 9]. Đối với BeVietnam, điều này là cần thiết vì hệ thống không chỉ có một chức năng duy nhất, mà nó phải trả lời liên tiếp ba câu hỏi cho người dùng: *nên đi đâu bây giờ, vì sao địa điểm đó đáng đi và khi đến đó thì nên làm gì*.

Trong thực tế làm đề tài, nhóm không chỉ phân rã bài toán ở mức tính năng, mà còn phân rã nó theo **user journey**, **dữ liệu**, **dịch vụ** và **ràng buộc kỹ thuật**. Đây là điểm quan trọng, vì nếu chỉ chia theo menu giao diện thì ta sẽ dễ bỏ sót các phụ thuộc cốt lõi của hệ thống. Ba lát cắt phân rã quan trọng nhất được trình bày trong Bảng 4.

Table 4: Các lát cắt phân rã chính trong quá trình phân tích BeVietnam

Mức phân rã	Cách nhóm phân tích	Kết quả rút ra
User journey	Nhóm chia hành trình của du khách thành chuỗi hành vi gồm mở ứng dụng, xem gợi ý, hiểu lý do đề xuất, chọn một địa điểm, mở chi tiết, bắt đầu storyline, di chuyển tới nơi, gửi capture và mở khóa nhiệm vụ tiếp theo [8].	Chuỗi hành vi này giúp nhóm xác định được đâu là vòng lặp sản phẩm cốt lõi, từ đó không để phạm vi trôi sang các chức năng thứ cấp như vlog tự động hay cộng đồng xã hội.
Dữ liệu	Nhóm chia hệ thống thành các lớp dữ liệu gồm dữ liệu POI, dữ liệu ngữ cảnh thời gian thực, dữ liệu tài khoản người dùng, dữ liệu tiến trình nhiệm vụ và dữ liệu capture. Việc phân rã này dẫn tới các mô hình như Place , User , Task , CaptureMetadata và các repository tương ứng trong codebase.	Hệ thống có được biên giới dữ liệu rõ ràng, từ đó thuận lợi hơn cho việc thiết kế schema, repository và luồng đồng bộ giữa backend với client.
Dịch vụ	Nhóm tách rõ backend nghiệp vụ, web client, mobile client và AI Core. Một số tài liệu cũ mô tả AI như trung tâm của toàn hệ thống, nhưng khi phân tích lại theo hướng phân rã, nhóm xem AI chỉ là một thành phần trong pipeline tổng thể [8].	Nhóm đi tới quyết định rằng backend phải giữ quyền kiểm soát product state, còn AI chỉ xử lý các khâu cần suy luận hoặc sinh nội dung.

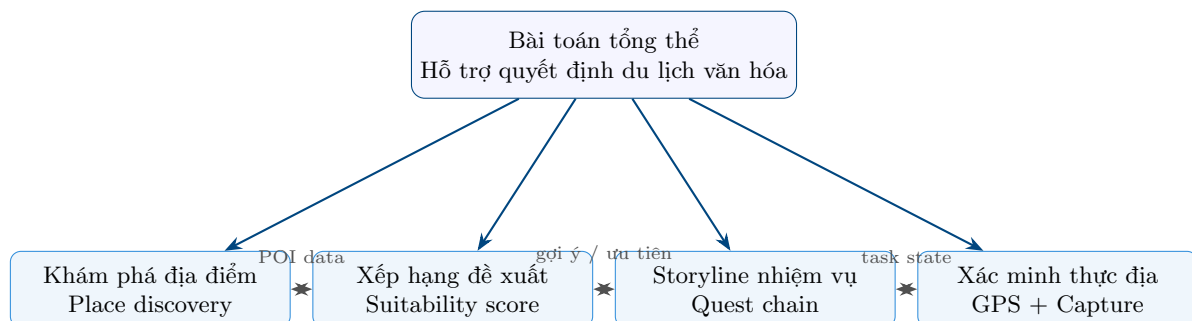


Figure 1: Sơ đồ phân rã bài toán BeVietnam thành bốn khối chức năng cốt lõi. Từ một bài toán hỗ trợ quyết định du lịch văn hóa, hệ thống được tách thành lớp dữ liệu địa điểm, lớp ra quyết định, lớp dẫn dắt hành trình và lớp xác minh thực địa.

Các lát cắt này cũng phản ánh hai lần nhóm phải sửa lại hướng đi. Bởi lẽ ban đầu nhóm đã triển khai AI-centric tức là một việc đều bỏ vô AI và giar quyết. Nhưng một bất cập diễn ra sau khi nhóm xây dựng

MVP theo ý tưởng đó, thấy được một điều là độ trễ giữa các công việc rất lâu thường mất khoảng 10s-20s. Điều này là rất tệ bởi lẽ, thay vì dành thời gian 10s-20s đó để chờ thì họ hoàn toàn có thể đi lên một nền tảng khác search và có được kết quả mong muốn. Đặc biệt là có giai đoạn nhóm đã phác thảo AI như “bộ não” trung tâm điều phối gần như mọi quyết định và xây dựng nó theo cơ chế Multi-Agent. Và rõ ràng khi chỉ với một Agent mà độ trễ đã rất lớn thì việc áp dụng nhiều agent với nhau với các vòng thì độ trễ càng tăng cao và làm cho trải nghiệm người dùng sẽ trở nên tệ hơn; và khi phân rã theo dịch vụ, nhóm nhận ra rằng thực tế có những giai đoạn ta hoàn toàn có thể tiền xử lý trước và sau đó đưa cho bên Backend kiểm soát và giữ lấy. Điều này sẽ khiến sản phẩm dễ kiểm soát và kiểm thử hơn và khi đó AI là một agent thì thực hiện tác vụ rất ít và sẽ có thể đem lại trải nghiệm thú vị hơn, mượt mà hơn. Tương tự, ở lát cắt dữ liệu, nhóm ban đầu định dùng Firebase cho cả xác thực lẫn lưu trữ, sau đó đổi sang PostgreSQL + MinIO + JWT để chủ động hơn về schema và phần backend tự vận hành và đặc biệt là chi phí vận hành sẽ tập trung vào một chỗ hơn thay vì là phân tán ở nhiều nơi. Từ các lát cắt đó, nhóm mới chốt bốn khối chức năng cốt lõi.

Table 5: Bốn khối chức năng cốt lõi sau bước decomposition

Khối chức năng	Đầu vào chính	Đầu ra / vai trò chính
Khám phá địa điểm	Dữ liệu POI đã chọn lọc, metadata văn hóa, hình ảnh, mô tả địa điểm.	Cung cấp lớp dữ liệu nền cho explore, place detail, feed và các feature phía sau.
Xếp hạng đề xuất	Vị trí người dùng, POI, weather, traffic, estimated crowdedness, rule scoring.	Trả về suitability score, explanation và thứ tự ưu tiên địa điểm tại thời điểm hiện tại.
Storyline nhiệm vụ	Place hiện tại, quest chain, progress state, nội dung văn hóa nền.	Sinh và điều phối chuỗi nhiệm vụ có mở khóa, có trạng thái và có bước tiếp theo rõ ràng.
Xác minh thực địa	GPS proximity, task hiện hành, capture metadata, tín hiệu hỗ trợ từ AI nếu cần.	Quyết định task có được đánh dấu hoàn thành hay không và lưu bằng chứng tương ứng.

Trước mắt đối với bài toán **khám phá địa điểm**. Tại đây hệ thống cần thu thập và lưu trữ thông tin về các điểm đến văn hóa, lịch sử, ẩm thực hay trải nghiệm địa phương, sau đó trả chúng về cho người dùng dưới dạng danh sách hoặc bản đồ. Nhưng khi đi vào phân tích kỹ hơn, nhóm nhận ra rằng “khám phá địa điểm” không chỉ là hiển thị tên địa danh và tọa độ trên bản đồ mà nó còn phải được chuẩn hóa dữ liệu POI để backend và hai client cùng hiểu một cấu trúc thống nhất. Song với đó nó phải mang bên mình một chiều sâu văn hóa của từng điểm đến thay vì chỉ còn lại các metadata khô cứng như địa chỉ hay giờ mở cửa. Từ đó, nó cho phép người dùng đi từ mức nhìn tổng quan trên feed hoặc map xuống mức đọc chi tiết về từng địa điểm mà không bị đứt mạch trải nghiệm. Đây là lý do thành phần này được gắn với các mô-đun **places**, **feed**, **place detail** và phần giao diện **Explore** trong cả mobile lẫn web [8]. Nói cách khác, đây là lớp nền giúp tất cả các quyết định sau đó có dữ liệu để hoạt động.



Figure 2: User journey mức cao của feature khám phá địa điểm.

Nhưng câu hỏi đặt ra là dữ liệu ở đâu ra? Để trả lời câu hỏi này, ta cần biết được khả năng của mô hình ngôn ngữ lớn tới đâu. Để tiết kiệm chi phí và phù hợp với budget của nhóm, nhóm đã chọn phương

án là tự self-hosting một mô hình mã nguồn nhẹ để đảm bảo được tốc độ của phản hồi. Tuy nhiên đối với các mô hình này chỉ rơi vào khoảng **3-8B** thì khả năng suy luận cũng như là kiến thức của nó về văn hóa cũng rất là hạn hẹp và thiếu thốn. Chính vì lẽ đó mà nhóm quyết định rằng, ta sẽ cần phải xây dựng được một kho tàng tri thức cho để giải quyết được bất cập trên. Lúc này bài toán sẽ được chuyển tiếp sang một bài toán mới đó là làm sao để có thể cung cấp được kho tàng tri thức văn hóa cho mô hình.

Tiếp đó là bài toán **xếp hạng và cá nhân hóa đề xuất**. Đây là phần quan trọng hơn, bởi hệ thống không chỉ hiển thị các địa điểm có sẵn mà còn phải quyết định địa điểm nào phù hợp hơn trong bối cảnh hiện tại. Điều này kéo theo các biến đầu vào như vị trí người dùng, khoảng cách, thời tiết, điều kiện giao thông, mức độ đông đúc và sở thích cá nhân. Nói cách khác, đây là một bài toán ra quyết định đa tiêu chí trong điều kiện dữ liệu thay đổi theo thời gian. Trong quá trình phân tích, nhóm xác định đây mới là trái tim thực sự của hệ thống, bởi nếu thành phần này yếu thì toàn bộ hệ thống sẽ quay lại trạng thái của một ứng dụng danh bạ du lịch thông thường. Do đó, thay vì chỉ nói “gợi ý điểm đến”, nhóm đã cụ thể hóa nó thành bài toán suitability score. Mỗi địa điểm phải được chấm điểm dựa trên cùng một logic chung, sau đó kết quả đó phải được tái sử dụng nhất quán cho cả feed lẫn bubble map. Hơn nữa, score này không được là một con số đen hộp. Nó còn phải sinh ra được lý do giải thích để người dùng hiểu vì sao một điểm đến được đẩy lên cao hay hạ xuống thấp tại thời điểm hiện tại. Đây chính là nơi tư duy tính toán buộc nhóm phải chuyển từ “cảm giác địa điểm này đáng đi” sang một mô hình quyết định có thể mô tả, kiểm tra và điều chỉnh được.



Figure 3: User journey mức cao của feature xếp hạng và cá nhân hóa đề xuất.

Kế đến là bài toán **dẫn dắt hành trình trải nghiệm** thông qua các nhiệm vụ văn hóa *storyline*. Hệ thống cần một tập nhiệm vụ hợp lý, có bối cảnh văn hóa và có thể mở khóa theo tiến độ. Như đã phân tích ở trên thay vì để mô hình ngôn ngữ sinh tự do từng nhiệm vụ theo mỗi lượt gọi, nhóm xây dựng trước một **kho nhiệm vụ** được sinh offline từ nguồn tri thức văn hóa, rồi nạp về client qua endpoint `/question-pool`. Toàn bộ kho này được trình bày dưới dạng một **Mission Path** trực quan (các màn hình *Storyline*, *TaskDetail*), trong đó mỗi nút là một nhiệm vụ. Nếu nhìn ở mức sản phẩm, đây là cơ chế biến quá trình tham quan từ bị động sang chủ động. Nhưng ở mức phân tích bài toán, đây là một bài toán điều hướng tiến trình rất rõ ràng: hệ thống phải biết người dùng đang ở bước nào, nhiệm vụ nào đã hoàn thành, nhiệm vụ nào đủ điều kiện mở khóa, và nội dung văn hóa nào nên xuất hiện ở bước tiếp theo. Điều này khiến storyline không còn là một đoạn content dài hay một mô-đun chat đơn giản, mà trở thành một cấu trúc tiến độ có trạng thái, có luật chuyển bước và có ràng buộc đầu vào. Việc tách kho nhiệm vụ (nội dung sinh offline) khỏi luồng suy luận của mô hình chính là cách nhóm giữ cho storyline ổn định và kiểm soát được, thay vì phụ thuộc vào AI sinh tự do theo runtime.



Figure 4: User journey mức cao của feature storyline nhiệm vụ.

Cuối cùng là bài toán **xác minh tương tác thực địa**. Nếu chỉ giao nhiệm vụ cho người dùng mà không có cơ chế xác minh, hệ thống sẽ mất đi tính tin cậy và cũng chẳng có gì lý thú và hấp dẫn. Bởi lẽ nếu

chỉ là nhiệm vụ dạng text bình thường thì người dùng đọc xong và cũng không muốn làm tiếp thì cũng không thể ép. Thay vào đó ta sẽ thêm một cơ chế xác minh để đảm bảo việc người dùng đã thật sự vượt qua task đó, đã tới đúng điểm đó để check-in thay vì chỉ là chụp đại một chỗ nào đó để qua. Do đó ta cần một cơ chế check-in, chụp ảnh và đánh giá việc hoàn thành nhiệm vụ. Đây là lý do BeVietnam có luồng **capture** và dịch vụ xác minh ảnh bằng AI, nhằm nối hành vi số trên ứng dụng với hành động thực tế ngoài đời. Hệ thống cần thực hiện các công việc bao gồm việc kiểm tra GPS proximity, lưu metadata của lần tương tác, gắn capture với đúng task và quyết định liệu trạng thái tiến trình có được cập nhật hay chưa. Để đánh giá ngữ nghĩa của ảnh, nhóm dùng một mô hình thị giác chấm điểm ảnh người dùng theo một **thang đánh giá tường minh** (rubric 0–1) so với chủ thể mà nhiệm vụ yêu cầu, trả về ba trạng thái *approved*, *needs_review* hoặc *rejected*. Điểm mấu chốt về mặt kiểm soát là kết quả của AI luôn nằm trong một khung nhất quán, để đảm bảo được sự toàn vẹn trong mỗi lần upload và kiểm thử: ngưỡng điểm quyết định trạng thái, và khi mô hình không sẵn sàng hoặc điểm nằm ở vùng nghi ngờ thì hệ thống chuyển sang *needs_review* chứ không tự ý chấp nhận. Nhờ vậy mà ta có thể giúp storyline gắn được với hành động thực địa thật thay vì chỉ là một lớp trò chơi hóa tồn tại trên màn hình.



Figure 5: User journey mức cao của feature xác minh thực địa.

2.4 Interact Diagram

Đối với các user journey ở trên mới vừa phân tích cũng như là từng feature thì nó chỉ nằm ở mức độ là rời rạc, và liệt kê. Tuy nhiên, trong một phiên sử dụng thực tế, người dùng đi xuyên qua nhiều feature liên tiếp, và mỗi thao tác lại kéo theo một chuỗi trao đổi giữa client, backend và các dịch vụ phía sau. Do đó nếu chỉ liệt kê và nói xuông thì ta sẽ không thể thấy rõ bức tranh động này, nhóm đã vẽ nên một phiên diễn hình bằng **sequence diagram** ở Hình 6, trong đó thể hiện đầy đủ vòng lặp sản phẩm: mở bản đồ thời gian thực, mở Mission Path và hoàn thành một nhiệm vụ qua xác minh ảnh.

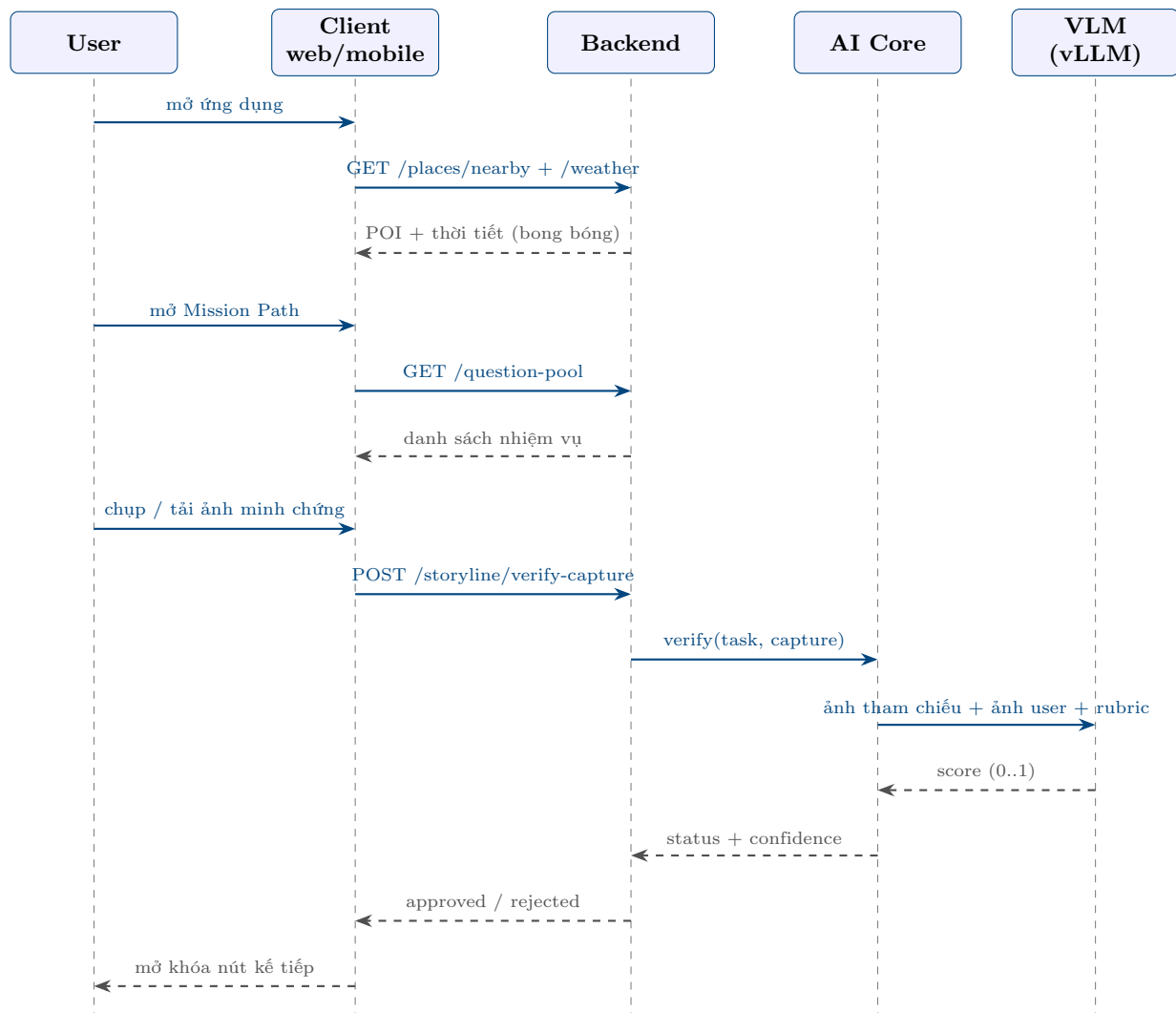


Figure 6: Sequence diagram cho một phiên sử dụng điển hình. Mũi tên liền là lời gọi, mũi tên đứt là phản hồi. Chỉ bước xác minh ảnh mới đi sâu xuống AI Core và mô hình thị giác (VLM); các bước còn lại do backend trực tiếp phục vụ.

Điểm đáng chú ý khi nhìn theo trục thời gian là phần lớn tương tác kết thúc ngay ở backend, chỉ riêng nhánh xác minh ảnh mới lan xuống AI Core rồi tới mô hình thị giác. Điều này phản ánh đúng quyết định thiết kế đã nêu: backend giữ vai trò điều phối, còn AI chỉ được kích hoạt ở đúng một mắt xích cần suy luận. Sơ đồ cũng cho thấy ranh giới trách nhiệm rõ ràng giữa các thành phần, từ đó giúp việc kiểm thử và xử lý lỗi trở nên dễ khoanh vùng hơn.

2.5 Data Model

Và sau khi ta đã phân rõ được hành vi của người dùng, các chức năng chính thì tiếp đó ta cần trả lời câu hỏi là *các thông tin ấy sẽ nhìn như thế nào bên trong hệ thống?*. Rõ ràng là máy tính sẽ không hiểu như cách con người chúng ta hiểu được, chính vì lẽ đó ta cần phải chuyển các tín hiệu thực tế thành các con số, các trường dữ liệu, các mô hình dữ liệu để ta có thể xử lý được bên trong máy tính. Một trong những lát cắt phân rã quan trọng nhất ở Bảng 4 là lát cắt **dữ liệu**. Tuy nhiên, việc liệt kê các nhóm dữ liệu mới chỉ là bước trung gian, bước quan trọng ở đây là biến các nhóm dữ liệu trừu tượng đó thành một **lược đồ cơ sở dữ liệu** cụ thể, có khóa chính, khóa ngoại và quan hệ rõ ràng, để backend có thể lưu trữ

và truy vấn một cách nhất quán. Bảng 6 trình bày ánh xạ này.

Table 6: Ánh xạ từ nhóm dữ liệu (decomposition) sang bảng dữ liệu

Nhóm dữ liệu	Bảng	Vai trò
Dữ liệu POI	places	Điểm đến văn hóa cốt lõi của hệ thống.
Tài khoản người dùng	users	Danh tính và xác thực người dùng.
Sở thích cá nhân	user_preferences	Tín hiệu cá nhân hóa cho feed (quan hệ 1–1 với người dùng).
Tiến trình & capture	captures	Bảng chứng thực địa, gắn người dùng với địa điểm và nhiệm vụ.
Kho nhiệm vụ	(tệp JSON)	Sinh offline, nạp trực tiếp về client nên không cần bảng riêng trong pilot.

Quan hệ giữa các bảng được mô tả trong sơ đồ thực thể – liên kết ở Hình 7. Có ba quan hệ chính: mỗi người dùng có tối đa một bản ghi sở thích (1–1), mỗi người dùng có thể tạo nhiều capture (1–*), và mỗi địa điểm cũng có thể nhận nhiều capture (1–*). Chính bảng **captures** là nơi nối hành động thực địa của người dùng với địa điểm và nhiệm vụ, nên nó mang hai khóa ngoại trở về **users** và **places**.

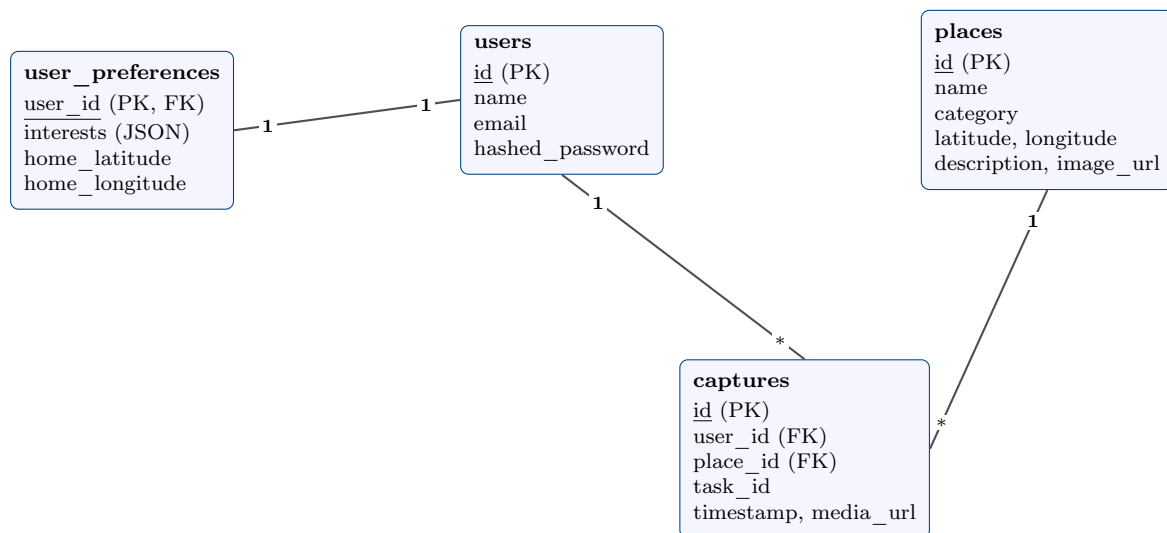


Figure 7: Sơ đồ thực thể – liên kết (ER) của BeVietnam. Bảng **captures** đóng vai trò bảng nối, mang khóa ngoại tới **users** và **places**; **user_preferences** mở rộng **users** theo quan hệ một – một để phục vụ cá nhân hóa.

Việc đi từ decomposition tới lược đồ dữ liệu này không chỉ là một thao tác kỹ thuật, mà còn là một bước trừu tượng hóa có chủ đích. Khi gom đúng các thuộc tính vào đúng thực thể và đặt quan hệ chính xác, hệ thống tránh được tình trạng dữ liệu trùng lặp hoặc mâu thuẫn giữa các client. Đồng thời, lược đồ gọn này cũng để ngỏ khả năng mở rộng: nếu sau pilot cần biến kho nhiệm vụ thành dữ liệu động hoặc thêm bảng tiến trình riêng cho storyline, ta chỉ việc bổ sung bảng mới trở về **users** mà không phải phá vỡ cấu trúc hiện có.

3 Tổng Quan Hệ Thống

Sau khi đã phân tích bài toán dưới góc nhìn Computational Thinking ở chương trước, ta tiến hành trình bày cái nhìn tổng thể về hệ thống BeVietnam. Mục tiêu của chương này không phải là đi sâu ngay

vào chi tiết kiến trúc kỹ thuật, mà là làm rõ hệ thống này đang phục vụ ai, các actor chính tương tác với hệ thống theo cách nào, hệ thống cung cấp những tính năng cốt lõi gì, và đâu là các điểm đổi mới làm cho BeVietnam khác với những ứng dụng du lịch thông thường. Nói cách khác, đây là chương giúp ta chuyển từ câu hỏi “ta đang giải bài toán gì” sang câu hỏi “hệ thống được tổ chức ra sao để giải bài to, bước quan trọng ở đây là Với một hệ thống du lịch thông minh như BeVietnam, stakeholder không chỉ đơn thuần là người trực tiếp bấm vào màn hình ứng dụng. Trên thực tế, hệ thống này chịu ảnh hưởng bởi nhiều nhóm khác nhau, từ người sử dụng cuối, nhóm phát triển cho tới những bên gián tiếp đánh giá tính đúng đắn của nội dung văn hóa và chất lượng pilot. Các stakeholder chính của hệ thống được tóm tắt trong Bảng 7.

Table 7: Main Stakeholders của hệ thống BeVietnam

Stakeholder	Vai trò đối với hệ thống	Mối quan tâm chính
Du khách	Là nhóm người dùng cuối trực tiếp sử dụng web hoặc mobile để xem gợi ý, so sánh địa điểm, mở storyline và gửi capture.	Muốn biết nên đi đâu vào lúc này, vì sao địa điểm đó đáng đi và khi đến nơi thì nên làm gì để có trải nghiệm văn hóa sâu hơn.
Nhóm phát triển dự án	Là nhóm xây dựng backend, AI Core, mobile app, web app và đồng thời vận hành pilot, kiểm thử và hoàn thiện báo cáo.	Cần một kiến trúc đủ rõ ràng để triển khai nhanh, kiểm thử được, có fallback và đủ ổn định cho Huế pilot.
Giảng viên / Hội đồng đánh giá	Là bên đánh giá đề tài ở góc độ học thuật, tính hợp lý của lời giải và mức độ hoàn thiện của sản phẩm.	Quan tâm tới việc hệ thống có giải quyết đúng Pain Point hay không, cách áp dụng tư duy tính toán có chặt chẽ hay không, và các quyết định kỹ thuật có được lập luận rõ ràng hay không.
Nguồn tri thức văn hóa và dữ liệu bối cảnh	Bao gồm các nguồn nội dung văn hóa đã được kiểm chứng, cùng các dịch vụ ngữ cảnh như weather, traffic hoặc crowd estimate được hệ thống sử dụng để ra quyết định.	Đòi hỏi hệ thống phải dùng dữ liệu một cách có nguồn gốc, không bịa nội dung văn hóa và không phụ thuộc hoàn toàn vào một tín hiệu duy nhất khi đề xuất địa điểm.

3.1 Main Actors

Nếu stakeholder mô tả các bên có lợi ích liên quan tới hệ thống, thì actor tập trung vào các nhóm thực sự tương tác với hệ thống trong luồng nghiệp vụ. Trong phạm vi pilot hiện tại, BeVietnam có ba actor chính như trong Bảng 8. Việc tách các actor này là rất cần thiết, vì mỗi actor có quyền hạn, mục tiêu và luồng thao tác khác nhau.

Table 8: Main Actors của hệ thống BeVietnam

Actor	Cách tương tác với hệ thống	Giới hạn / phạm vi thao tác
Traveler	Mở ứng dụng, xem feed, xem bubble map, đọc place detail, bắt đầu storyline, gửi capture và theo dõi tiến trình khám phá.	Là actor trung tâm của toàn bộ sản phẩm. Trong pilot Huế, actor này bao gồm cả du khách nội địa và quốc tế [8].
Project Team / Operator	Seed dữ liệu, quan sát log, kiểm thử các flow chính, kiểm tra AI output, chuẩn bị nội dung pilot và đối chiếu hành vi hệ thống với đặc tả.	Không phải actor của trải nghiệm người dùng cuối, nhưng lại là actor quan trọng để hệ thống có thể chạy được trong bối cảnh pilot thực tế.
Future Admin / Moderator	Xem xét dữ liệu, kiểm tra nội dung sinh tự động, theo dõi capture hoặc nội dung bị gán cờ, và quản lý chất lượng hệ thống khi mở rộng về sau.	Chưa phải thành phần trọng tâm của Huế pilot, nhưng đã được nhận diện từ sớm để tránh việc kiến trúc hiện tại khóa mất khả năng quản trị trong tương lai [8].

3.2 Core Features

Sau khi khóa stakeholder và actor, bước tiếp theo là xác định những tính năng cốt lõi thật sự định nghĩa giá trị của hệ thống. Trong BeVietnam, đây không phải là danh sách càng dài càng tốt. Ngược lại, nhóm chủ động giữ lại một số lượng feature đủ tinh gọn để bảo vệ trọng tâm của pilot Huế. Các core feature hiện tại được trình bày trong Bảng 9.

Table 9: List of Core Features của BeVietnam

Core Feature	Chức năng cốt lõi	Vai trò trong pilot
Curated Place Discovery	Hiển thị các điểm đến văn hóa đã được chọn lọc, có thông tin nền rõ ràng và có thể mở chi tiết trên cả web lẫn mobile.	Nền dữ liệu cơ sở
Dynamic Recommendation Feed	Trả về danh sách địa điểm được xếp hạng theo bối cảnh hiện tại cùng phần giải thích lý do đề xuất.	Giá trị chính của sản phẩm
Realtime Bubble Map	Hiển thị các POI thời gian thực (Foursquare) quanh vị trí người dùng trong bán kính 3km, kích thước bong bóng co giãn theo khoảng cách và kèm thời tiết khu vực.	So sánh trực quan
Storyline Mission Path	Trình bày kho nhiệm vụ văn hóa dưới dạng lộ trình kiểu Duolingo, mỗi nút là một nhiệm vụ nạp từ question pool, mở khóa tuần tự theo tiến độ.	Trải nghiệm đào sâu
Vision Capture Verification	Chấm điểm ảnh minh chứng bằng mô hình thị giác theo rubric tường minh (so với ảnh tham chiếu), quyết định mở khóa nhiệm vụ.	Nổi trải nghiệm số với thực địa
Bilingual Experience	Hỗ trợ tiếng Việt và tiếng Anh cho các bề mặt tương tác chính trong pilot.	Phù hợp với đối tượng người dùng mục tiêu

3.3 Explanation of Core Features

Mỗi core feature trong BeVietnam được tạo ra để giải một Pain Point cụ thể chứ không phải chỉ để làm cho hệ thống có nhiều chức năng hơn. Vì vậy, ở phần này ta không chỉ mô tả feature theo tên gọi, mà sẽ lần lượt trình bày ba câu hỏi cho từng feature: nó được tạo ra để giải quyết vấn đề nào, nó nhận đầu

vào và tạo đầu ra ra sao, và cơ chế vận hành chính của nó là gì.

3.3.1 Curated Place Discovery

Pain Point đầu tiên của người dùng là thông tin du lịch bị phân tán trên nhiều nền tảng khác nhau. Người dùng có thể tìm thấy tên địa điểm trên Google Maps, đọc một vài câu chuyện trên blog, rồi lại phải chuyển sang mạng xã hội để xem hình ảnh thực tế. Điều này khiến quá trình khám phá ban đầu rất rời rạc và thiếu chiều sâu văn hóa. Vì vậy, feature **Curated Place Discovery** được xây dựng để tạo ra một lớp dữ liệu điểm đến có chọn lọc và có cấu trúc thống nhất ngay trong hệ thống.

Đầu vào của feature này là tập dữ liệu POI đã được nhóm chọn lọc, bao gồm tên địa điểm, loại hình, tọa độ, mô tả, hình ảnh, thông tin văn hóa và một số metadata phục vụ cho các bước xử lý phía sau. Đầu ra của nó là danh sách địa điểm hoặc màn hình place detail mà người dùng có thể truy cập từ web và mobile. Đây là lớp đầu ra nền tảng, bởi tất cả các feature như feed, bubble map hay storyline đều cần một tập điểm đến đáng tin cậy để hoạt động.

Với người dùng không chuyên về kỹ thuật, có thể hiểu feature này như một “lớp biên tập số” của toàn hệ thống. Thay vì ném cho du khách một danh sách dài các điểm đến lẫn lộn về chất lượng, BeVietnam cố gắng chọn ra những nơi thật sự có ý nghĩa với pilot, rồi đóng gói lại thành một trải nghiệm khám phá mạch lạc. Khi người dùng mở ứng dụng, họ không chỉ thấy tên và ảnh của một địa điểm, mà còn thấy vì sao địa điểm đó đáng để đọc thêm, đáng để lưu ý, và đáng để bước sang các feature sâu hơn như feed hay storyline.

Với người đọc kỹ thuật, feature này chính là tầng chuẩn hóa dữ liệu đầu vào cho toàn bộ sản phẩm. Backend cần tổ chức POI theo schema thống nhất, để cùng một thực thể **Place** có thể được dùng lại cho explore, place detail, feed, map và storyline mà không bị lệch nghĩa giữa các client. Một điểm quan trọng là lớp dữ liệu này không chỉ chứa thông tin hiển thị, mà còn mang theo metadata phục vụ scoring và liên kết tới các lớp nội dung văn hóa phía sau. Vì vậy, Curated Place Discovery không phải chỉ là feature “xem địa điểm”, mà là tầng nền dữ liệu giúp mọi feature còn lại có thể hoạt động ổn định và nhất quán.

3.3.2 Dynamic Recommendation Feed

Pain Point thứ hai là người dùng không chỉ cần biết “có những nơi nào”, mà họ cần biết “nên đi đâu vào lúc này”. Nếu hệ thống chỉ liệt kê các địa điểm tĩnh thì nó không khác nhiều so với một danh bạ du lịch. Vì vậy, feature **Dynamic Recommendation Feed** được xây dựng để đưa ra quyết định xếp hạng trong bối cảnh hiện tại thay vì chỉ hiển thị dữ liệu thô.

Đầu vào của feature này gồm vị trí người dùng, tập POI có sẵn, các yếu tố ngữ cảnh như weather, traffic, estimated crowdedness và các tham số ưu tiên đã được định nghĩa trong mô hình scoring. Đầu ra của nó là danh sách các địa điểm được xếp hạng theo mức độ phù hợp, kèm theo factor score và phần giải thích ngắn gọn để người dùng hiểu vì sao địa điểm đó được đưa lên hoặc bị hạ xuống.

Với người dùng, đây là feature gần nhất với giá trị cốt lõi của sản phẩm. Người dùng không cần tự mình ngồi so từng địa điểm, đoán xem trời mưa có nên đi hay không, hay tự cân nhắc giữa khoảng cách và giá trị văn hóa. Họ chỉ cần mở feed và nhìn thấy một câu trả lời đã được hệ thống tổng hợp sẵn: nơi nào đang phù hợp hơn, nơi nào kém phù hợp hơn, và lý do của sự khác biệt đó là gì. Chính phần explanation làm cho feature này khác với một bảng xếp hạng thuần túy, bởi người dùng không chỉ nhận kết quả, mà còn hiểu được lập luận phía sau kết quả.

Ở góc nhìn kỹ thuật, Dynamic Recommendation Feed là nơi mô hình quyết định của BeVietnam được

triển khai rõ nhất. Backend phải lấy tập POI khả dụng, thu thập các tín hiệu ngữ cảnh tại thời điểm hiện tại, chuẩn hóa các factor, rồi tính *suitability score* cho từng điểm đến. Sau bước tính score, hệ thống còn phải giữ lại factor score, fallback flag và explanation để trả về cho client. Điều này có nghĩa là feed không chỉ là một API trả danh sách, mà là một pipeline gồm thu thập dữ liệu, tính toán, giải thích và trình bày. Nếu pipeline này không chặt, hệ thống sẽ nhanh chóng rơi lại thành một ứng dụng danh bạ có thêm một ít văn bản trang trí.

3.3.3 Realtime Bubble Map

Pain Point thứ ba là ngay cả khi đã có danh sách gợi ý, người dùng vẫn cần một cách so sánh nhanh các địa điểm trong không gian xung quanh. Nếu chỉ đọc feed tuần tự từ trên xuống dưới, họ sẽ khó hình dung tương quan về vị trí và mức độ phù hợp giữa các điểm đến. Vì vậy, feature **Realtime Bubble Map** được xây dựng để biến quá trình so sánh đó thành một trải nghiệm trực quan hơn, dựa trên dữ liệu thời gian thực.

Đầu vào của feature này gồm vị trí GPS thực của người dùng và một bán kính quét cố định (3km). Hệ thống truy vấn các POI thực tế quanh người dùng qua nguồn Foursquare và thu thập thời tiết khu vực (nhiệt độ, chỉ số UV ước lượng, lượng mưa). Đầu ra là các bong bóng hiển thị trên bản đồ, trong đó kích thước mỗi bong bóng co giãn theo khoảng cách thực tế từ địa điểm tới người dùng (càng gần càng lớn), kèm một chip thời tiết của khu vực.

Với người dùng, bubble map giải quyết một nhu cầu rất tự nhiên khi đi du lịch: ngoài việc biết nơi nào tốt hơn, họ còn muốn nhìn thấy các lựa chọn đó đang nằm ở đâu quanh mình. Nếu feed giúp người dùng suy nghĩ theo trục “đáng đi hay không đáng đi”, thì bubble map giúp họ suy nghĩ thêm theo trục “gần hay xa” và “nằm trong không gian di chuyển của mình hay không”. Nhờ vậy, quyết định du lịch trở nên trực quan hơn rất nhiều, đặc biệt trong bối cảnh người dùng đang đứng giữa nhiều lựa chọn gần nhau.

Với người đọc kỹ thuật, điểm quan trọng nhất của feature này là tính *thời gian thực* và việc khóa nguồn dữ liệu về một mối. Ở bản đầu nhóm chỉ dùng tập POI tuyển chọn thủ công, nhưng một danh sách tĩnh không bao giờ phản ánh đúng những gì đang ở quanh người dùng, nên nhóm bổ sung nguồn Foursquare thời gian thực. Khóa Foursquare được giữ ở backend, còn client chỉ truyền tọa độ và nhận về danh sách POI đã được phân loại theo nhóm (ẩm thực, lưu trú, văn hóa, lịch sử...). Cả web lẫn mobile dùng chung một endpoint `/places/nearby`, nhờ đó hai client cho ra cùng một lớp bong bóng nhất quán. Việc co giãn theo khoảng cách được tính phía client bằng công thức haversine từ vị trí GPS, nên bản đồ luôn phản ánh đúng bối cảnh hiện tại của người dùng thay vì một danh sách tĩnh.

3.3.4 Storyline Mission Path

Pain Point tiếp theo là sau khi đã chọn được một địa điểm, người dùng vẫn thường không biết nên làm gì tại đó để thật sự hiểu được giá trị văn hóa của nơi mình đến. Nếu hệ thống chỉ dừng lại ở việc khuyên người dùng “hãy ghé thăm nơi này”, thì trải nghiệm khám phá vẫn còn khá thụ động. Vì vậy, feature **Storyline Mission Path** được xây dựng để trả lời câu hỏi *khi đã đến đó thì nên làm gì tiếp theo*, trình bày toàn bộ nhiệm vụ dưới dạng một lộ trình trực quan kiểu Duolingo.

Đầu vào của feature này là kho nhiệm vụ (question pool) đã được sinh offline từ nội dung văn hóa, cùng trạng thái tiến trình của người dùng. Đầu ra của nó là một lộ trình các nút nhiệm vụ có thứ tự, mỗi nút mang nội dung hướng dẫn, bối cảnh văn hóa, điều kiện mở khóa và yêu cầu minh chứng. Người dùng đi dọc lộ trình, mở từng nút để xem chi tiết và nộp ảnh minh chứng nhằm mở khóa nút kế tiếp.

Với người dùng, feature này làm cho chuyến đi có cảm giác được dẫn dắt thay vì chỉ tự do đọc rồi tự đoán xem mình nên quan sát gì. Một nhiệm vụ được thiết kế tốt sẽ khiến người dùng chú ý tới một chi tiết kiến trúc, một dấu vết lịch sử hay một hành động văn hóa cụ thể mà nếu không có storyline họ rất dễ bỏ qua. Vì vậy, giá trị của feature này không nằm ở số lượng task, mà nằm ở việc nó biến kiến thức văn hóa thành hành động có thể thực hiện tại địa điểm thật.

Từ góc nhìn kỹ thuật, Storyline Mission Path là một bài toán quản lý tiến trình có trạng thái. Hệ thống phải biết người dùng đang ở nút nào, nút nào đã hoàn thành, điều kiện nào cho phép mở khóa nút tiếp theo và nội dung nào cần được trả ra ở bước hiện tại. Vai trò của AI được tách bạch rất rõ: nội dung nhiệm vụ được **sinh trước offline** vào question pool dưới dạng có cấu trúc (tiêu đề, mô tả, giải nghĩa văn hóa, yêu cầu minh chứng), còn tại runtime client chỉ nạp pool, dựng lộ trình và quản lý trạng thái mở khóa cục bộ. Hướng này không có ngay từ đầu: ban đầu nhóm định để AI sinh từng nhiệm vụ ngay lúc người dùng chơi, nhưng khi thử thì kết quả thiếu ổn định (định dạng lệch, nội dung lúc được lúc không) và phụ thuộc nặng vào hạn mức gọi mô hình, nên nhóm chuyển sang dựng sẵn kho nhiệm vụ offline rồi nạp về. Nhờ vậy, feature này vừa là lớp kể chuyện cho người dùng, vừa là một state machine ổn định cho hệ thống, không phụ thuộc vào việc gọi mô hình ngôn ngữ theo từng lượt tương tác.

3.3.5 Vision Capture Verification

Nếu storyline chỉ yêu cầu người dùng bấm nút hoàn thành thì toàn bộ lớp trải nghiệm thực địa sẽ mất đi độ tin cậy. Pain Point ở đây là hệ thống cần một cách xác minh rằng người dùng thật sự đã tới nơi và chụp đúng chủ thể được yêu cầu. Vì vậy, feature **Vision Capture Verification** được xây dựng để nối trạng thái số của nhiệm vụ với hành vi thực tế ngoài đời.

Đầu vào của feature này gồm nhiệm vụ hiện hành (kèm chủ thể cần chụp), ảnh người dùng nộp lên và một ảnh tham chiếu chuẩn của chủ thể đó. Đầu ra của nó là một trạng thái xác minh kèm điểm tin cậy: *approved*, *needs_review* hay *rejected*, cùng lý do ngắn gọn, từ đó quyết định nút nhiệm vụ có được mở khóa hay không.

Với người dùng, feature này làm cho việc hoàn thành nhiệm vụ có ý nghĩa thật sự. Nếu không có một cơ chế xác minh tối thiểu, storyline sẽ rất dễ trở thành một chuỗi nút bấm “đã xong” mà không còn gắn với hành động thực tế nào. Khi hệ thống yêu cầu GPS proximity và capture, người dùng có cảm giác rằng tiến trình của mình được gắn với trải nghiệm ngoài đời, không chỉ với thao tác trên màn hình.

Ở góc nhìn kỹ thuật, đây là nơi hệ thống nối state số với bằng chứng thực địa thông qua một mô hình thị giác. Bản đầu tiên của phần này thực ra rất sơ sài: hệ thống chỉ kiểm tra “có ảnh hay không” rồi duyệt, nên khi nhóm thử tải lên một ảnh không liên quan thì nó vẫn được chấp nhận ở mức tin cậy cao. Chính lỗi đó buộc nhóm làm lại theo hướng chấm điểm thật. Nhóm cũng từng dùng Gemini cho bước này, nhưng vì gặp giới hạn hạn mức nên chuyển sang tự host một mô hình thị giác trên vLLM. Ở phiên bản hiện tại, khi nhận ảnh, AI Core tìm (hoặc tải về và lưu cache) một ảnh tham chiếu của chủ thể nhiệm vụ, rồi yêu cầu mô hình thị giác so sánh ảnh người dùng với ảnh tham chiếu theo một rubric tường minh và trả về điểm 0–1. Điểm này được ánh xạ sang ba trạng thái qua các ngưỡng cố định. Điểm mấu chốt về kiểm soát là kết quả của AI không bao giờ là một hộp đen tùy tiện: rubric, ngưỡng và cơ chế *needs_review* (khi mô hình không sẵn sàng hoặc điểm nằm ở vùng nghi ngờ) giữ cho quyết định luôn nằm trong một khung minh bạch. Do đó, feature này vừa mang tính trải nghiệm, vừa là lớp kiểm soát nghiệp vụ quan trọng, và cũng là tính năng duy nhất bắt buộc phải dùng tới mô hình AI tự host tại thời gian thực.

3.3.6 Bilingual Experience

Vì pilot hướng đến cả du khách nội địa và quốc tế, Pain Point cuối cùng ở đây là rào cản ngôn ngữ. Một sản phẩm có nội dung văn hóa tốt nhưng không truyền đạt được nhất quán cho hai nhóm người dùng mục tiêu thì vẫn chưa hoàn thành nhiệm vụ của nó. Do đó, feature **Bilingual Experience** được đưa vào như một lớp cốt lõi của hệ thống thay vì chỉ là phần phụ trợ về giao diện.

Đầu vào của feature này là toàn bộ nội dung và nhãn hiển thị trên các bề mặt chính của hệ thống, cùng với trạng thái ngôn ngữ mà người dùng đã chọn. Đầu ra là giao diện và nội dung được hiển thị nhất quán theo tiếng Việt hoặc tiếng Anh.

Với người dùng, feature này quyết định việc họ có thật sự hiểu được giá trị văn hóa mà hệ thống đang cố truyền tải hay không. Nếu nội dung chỉ được viết tốt bằng một ngôn ngữ, còn ngôn ngữ còn lại chỉ là bản dịch sơ sài, thì trải nghiệm của hai nhóm du khách sẽ bị lệch rất mạnh. Vì vậy, feature song ngữ không phải là một lớp trang trí giao diện, mà là điều kiện để sản phẩm giữ được cùng một thông điệp cho cả traveler nội địa lẫn quốc tế.

Từ góc nhìn kỹ thuật, feature này yêu cầu cả client lẫn lớp nội dung phải phối hợp chặt. Client chịu trách nhiệm chuyển đổi nhãn hiển thị, điều hướng giao diện và trạng thái ngôn ngữ, trong khi backend và dữ liệu nội dung phải bảo đảm rằng các phần mô tả cốt lõi có thể được cung cấp nhất quán ở cả tiếng Việt lẫn tiếng Anh. Điều này đặc biệt quan trọng trong một hệ thống như BeVietnam, bởi rất nhiều giá trị của sản phẩm nằm ở phần diễn giải văn hóa chứ không chỉ ở vị trí địa lý của địa điểm.

3.4 Pipeline Tổng Thể Hệ Thống

Sau khi đã đi qua từng feature một cách riêng lẻ, phần này gom các thành phần đó lại thành một bức tranh chung để thấy chúng phối hợp ra sao trong một luồng chạy thật. Điểm quan trọng nhất khi nhìn ở mức tổng thể là hệ thống được chia thành ba lớp tách bạch: **Frontend Layer** (hai client web và mobile), **Backend Layer** (FastAPI cùng các data store và các nguồn dữ liệu bên ngoài) và **vLLM Layer** (mô hình thị giác tự host). Cách chia lớp này phản ánh đúng một quyết định thiết kế mà nhóm đã chốt từ sớm: backend giữ vai trò điều phối và product state, còn AI chỉ là một thành phần được gọi tới khi thật sự cần, chứ không phải trung tâm của hệ thống. Toàn bộ pipeline được trình bày trong Hình 8.

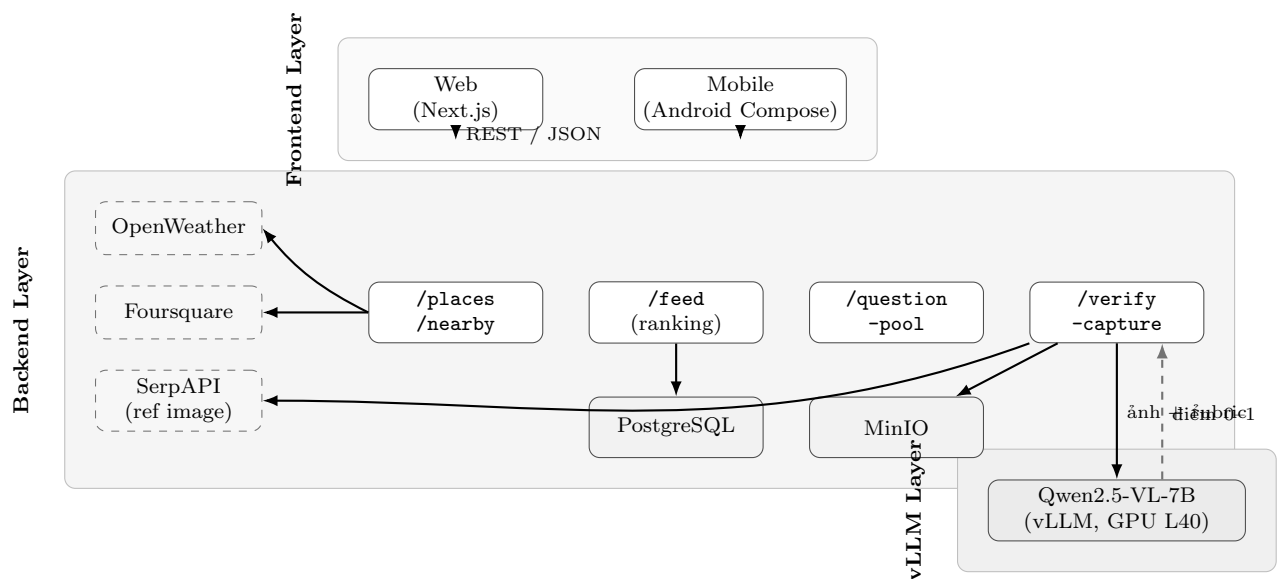


Figure 8: Pipeline tổng thể của BeVietnam, chia thành ba lớp Frontend, Backend và vLLM. Hai client dùng chung tập endpoint của backend; backend điều phối các nguồn dữ liệu ngoài (Foursquare, OpenWeather, SerpAPI) và data store (PostgreSQL, MinIO); riêng lớp vLLM chỉ được gọi tại endpoint `/verify-capture`.

Nhìn vào Hình 8 có thể thấy ba điều phản ánh đúng cách nhóm đã xây hệ thống. Trước hết, hai client web và mobile không nói chuyện trực tiếp với nhau hay với mô hình AI, mà cùng đi qua một tập endpoint thống nhất của backend, nhờ đó hai bề mặt cho ra trải nghiệm nhất quán. Tiếp đến, các nguồn dữ liệu ngoài (Foursquare, OpenWeather, SerpAPI) đều bị khóa sau backend thay vì lộ ra client, vừa để giữ khóa API an toàn vừa để chuẩn hóa dữ liệu trước khi trả về. Sau cùng, lớp vLLM nằm tách hẳn và chỉ có duy nhất `/verify-capture` chạm tới, đúng với kết luận ở phần trước rằng xác minh ảnh là tính năng duy nhất bắt buộc dùng mô hình AI tự host tại thời gian thực.

3.5 Innovation Highlights

Điểm đổi mới của BeVietnam không nằm ở việc sử dụng AI như một nhãn mác chung chung, mà nằm ở cách hệ thống kết hợp nhiều thành phần công nghệ và nghiệp vụ để giải quyết đúng Pain Point của người đi du lịch. Các đổi mới nổi bật nhất của hệ thống được tóm tắt trong Bảng 10.

Table 10: Innovation Highlights của BeVietnam

Điểm đổi mới	Ý nghĩa đối với hệ thống
Recommendation theo ngữ cảnh thời gian thực	Khác với nhiều ứng dụng du lịch chỉ hiển thị các điểm đến phổ biến, BeVietnam cố gắng trả lời câu hỏi điểm đến nào phù hợp <i>ngay lúc này</i> bằng cách kết hợp cultural value với weather, traffic, distance và estimated crowdedness.
Explainable ranking	Hệ thống không chỉ trả về kết quả xếp hạng, mà còn cố gắng giải thích vì sao địa điểm được đề xuất hoặc bị hạ ưu tiên. Điều này làm cho recommendation trở nên minh bạch hơn và phù hợp hơn với định hướng học thuật của đề tài.
Bản đồ POI thời gian thực	Thay vì một danh sách tĩnh, bubble map truy vấn các địa điểm thực tế quanh vị trí GPS của người dùng và co giãn bong bóng theo khoảng cách thật. Hai client web và mobile dùng chung một nguồn dữ liệu để giữ trải nghiệm nhất quán.
Storyline gắn với hành động thực địa	BeVietnam không dừng ở việc kể chuyện văn hóa, mà biến nội dung đó thành chuỗi nhiệm vụ có thể thực hiện tại địa điểm thật. Điều này tạo ra cầu nối giữa nội dung số và trải nghiệm thực tế tại điểm đến.
Xác minh ảnh bằng mô hình thị giác tự host	Thay vì để AI chấm bữa hoặc duyệt thủ công, BeVietnam dùng một mô hình thị giác (Qwen2.5-VL tự host trên vLLM) chấm ảnh theo rubric tường minh, có ảnh tham chiếu và ngưỡng quyết định, kèm cơ chế chuyển duyệt thủ công khi nghi ngờ.
Kết hợp học văn hóa với quyết định du lịch	Nhiều ứng dụng chỉ mạnh về thông tin địa điểm hoặc chỉ mạnh về nội dung kể chuyện. BeVietnam cố gắng kết hợp hai lớp giá trị này: giúp người dùng đưa ra quyết định đi đâu, đồng thời giúp họ hiểu sâu hơn về nơi họ tới.

Do đó ta có thể thấy rằng, tổng quan hệ thống BeVietnam không được xây dựng như một tập hợp tính năng rời rạc, mà được tổ chức thành một lời giải có trọng tâm rất rõ. Traveler là actor trung tâm, dynamic recommendation feed và bubble map là giá trị chính của pilot, còn storyline cùng capture verification là lớp trải nghiệm đào sâu giúp hệ thống khác biệt với các ứng dụng du lịch tra cứu thông thường. Từ nền tảng tổng quan này, chương tiếp theo có thể đi sâu hơn vào kiến trúc kỹ thuật, data flow và cách các thành phần backend, AI và client phối hợp với nhau để triển khai các feature trên.

4 Pattern Recognition

Sau khi phân rã bài toán, bước tiếp theo của tư duy tính toán là tìm ra những **pattern** lặp đi lặp lại giữa các bài toán con. Điều này có ý nghĩa quan trọng, vì nhiều khi các vấn đề trông có vẻ khác nhau ở bề mặt nhưng thực chất lại có chung một cấu trúc xử lý bên dưới. Khi nhận ra các quy luật đó, ta có thể tái sử dụng thiết kế, dữ liệu và logic thay vì xây dựng mọi thứ theo cách rời rạc [6, 7].

Đối với BeVietnam, ta nhận ra có ba mẫu lặp rất rõ, và mỗi mẫu lặp này đều dẫn tới một quyết định thiết kế cụ thể.

Đầu tiên là mẫu **context-aware decision making**. Dù là feed gợi ý địa điểm, bubble map hay lựa chọn nhiệm vụ kế tiếp trong storyline thì tất cả đều dựa trên cùng một câu hỏi nền tảng: với trạng thái hiện tại của người dùng và môi trường xung quanh, hệ thống nên ưu tiên điều gì trước. Điều này có nghĩa là nhiều tính năng khác nhau thực ra cùng chia sẻ một lớp tư duy ra quyết định theo ngữ cảnh. Từ pattern này, nhóm đi tới quyết định rằng feed và bubble map không được phép dùng hai công thức khác nhau, mà phải dùng cùng một lõi suitability score để tránh việc hai bề mặt hiển thị đưa ra hai thông điệp mâu thuẫn. Nếu không nhận ra pattern này từ sớm, rất dễ xảy ra tình huống mỗi màn hình được xây theo một logic riêng: feed xếp theo mức hấp dẫn nội dung, bản đồ phóng to theo khoảng cách, còn storyline lại ưu

tiên theo cảm hứng riêng của AI. Khi đó người dùng sẽ không còn thấy một hệ thống thống nhất, mà chỉ thấy nhiều tính năng đứng cạnh nhau nhưng không nói cùng một ngôn ngữ quyết định.

Kế đến là mẫu **retrieve** → **explain** → **present**. Trong khám phá địa điểm, hệ thống truy xuất dữ liệu về điểm đến, sau đó giải thích giá trị của địa điểm đó và cuối cùng trình bày nó thành card, feed hoặc bong bóng bản đồ. Trong storyline, hệ thống cũng truy xuất bối cảnh văn hóa, sinh ra mô tả nhiệm vụ và cuối cùng hiển thị nó cho người dùng dưới dạng các bước hành động. Điều này cho thấy dù giao diện đầu ra khác nhau, cấu trúc xử lý thông tin vẫn rất tương đồng. Từ đó mà nhóm thiết kế AI theo hướng lấy dữ liệu văn hóa có nguồn gốc trước, sau đó mới sinh phần giải thích hoặc nhiệm vụ bằng mô hình ngôn ngữ [8]. Điều này có ý nghĩa thực tế rất lớn, vì nó ngăn hệ thống rơi vào cách làm phổ biến nhưng rất yếu là đưa prompt chung chung cho mô hình rồi hy vọng mô hình “sẽ tự biết” nên nói gì. Thay vào đó, từng khối tính năng đều phải đi qua cùng một cấu trúc: lấy đúng dữ liệu nền, sinh nội dung dựa trên dữ liệu đó, rồi mới trình bày ra giao diện theo hình thức phù hợp.

Cuối cùng là mẫu **verify** → **update state**. Sau mỗi hành động quan trọng của người dùng, ví dụ như hoàn thành một nhiệm vụ hay gửi một bức ảnh minh chứng, hệ thống đều phải kiểm tra dữ liệu đầu vào, xác minh tính hợp lệ rồi mới cập nhật trạng thái tiến trình. Mẫu này lặp lại rất nhiều trong backend, AI service và giao diện người dùng, từ đăng nhập, gửi capture cho tới hoàn thành task. Pattern này dẫn nhóm tới yêu cầu là mọi hành động cập nhật tiến trình phải đi qua lớp validation và không để AI ghi trực tiếp vào product state. Đây là quyết định quan trọng ở giai đoạn pilot, vì nếu bỏ qua lớp kiểm tra này thì các tính năng liên quan đến tài khoản, tiến trình storyline và capture sẽ rất dễ rơi vào tình trạng không nhất quán, đặc biệt khi hệ thống còn đang dùng nhiều nguồn dữ liệu và nhiều bề mặt client khác nhau.

Việc nhận diện các mẫu lặp này có hậu quả kỹ thuật rất rõ ràng. Trước hết, nó giúp nhóm chuẩn hóa cách thiết kế API, vì nhiều luồng có thể dùng chung cấu trúc request/response và cách quản lý trạng thái. Kế đó, nó giúp nhóm chuẩn hóa cách viết service và repository, bởi dù dữ liệu thuộc về **place**, **feed** hay **task**, ta vẫn có thể tổ chức theo các tầng rất giống nhau. Sau cùng, nó tạo tiền đề để AI không hoạt động như một hộp đen độc lập, mà chỉ là một mắt xích trong một pipeline có pattern rõ ràng: nhận dữ liệu đầu vào, suy luận trong phạm vi kiểm soát và trả về output có cấu trúc.

5 Abstraction

Nếu decomposition giúp ta chia nhỏ vấn đề, còn pattern recognition giúp ta tìm ra quy luật lặp, thì **abstraction** lại giúp ta loại bỏ các chi tiết không thiết yếu để giữ lại cấu trúc bản chất của bài toán. Đây là bước quan trọng trong phát triển phần mềm, vì thế giới thực luôn rất phức tạp, còn máy tính thì chỉ làm việc hiệu quả khi bài toán được biểu diễn dưới dạng các mô hình dữ liệu và các quan hệ rõ ràng [5, 6, 9].

Trong BeVietnam, bài toán du lịch ngoài đời thực có thể chứa vô số chi tiết khác nhau như cảm xúc cá nhân của du khách, mức độ nổi tiếng trên mạng xã hội, độ đẹp của ảnh chụp hay rất nhiều yếu tố khó định lượng khác. Nhưng để hệ thống có thể vận hành, ta buộc phải trừu tượng hóa các thành phần đó thành các đối tượng và thuộc tính cốt lõi, tóm tắt trong Định nghĩa 5.1.

Định nghĩa 5.1: Bốn Lớp Trừu Tượng Cốt Lõi

Hệ thống quy bài toán du lịch về bốn lớp trừu tượng:

1. **Place** — địa điểm với tọa độ, loại hình, mô tả và metadata phục vụ xếp hạng.
2. **score** — mức độ phù hợp tại thời điểm hiện tại, một đại lượng tính được từ các yếu tố ngữ cảnh.
3. **Task** — một nút trong hành trình, có thứ tự, điều kiện mở khóa và tiêu chí hoàn thành.
4. **Capture** — bằng chứng thực địa, ánh xạ hành động ngoài đời của người dùng thành dữ liệu xác minh được.

Đầu tiên, ta trừu tượng hóa khái niệm *địa điểm du lịch* thành thực thể **Place**. Một **Place** không còn là toàn bộ trải nghiệm phong phú ngoài đời, mà được biểu diễn bằng các thuộc tính như tên, tọa độ, loại hình, mô tả, hình ảnh, thông tin văn hóa và các chỉ số phục vụ xếp hạng. Nhờ vậy máy tính mới có thể so sánh, lọc và sắp xếp các địa điểm một cách tường minh. Trong quá trình làm đề tài, lớp trừu tượng này còn giúp nhóm tách bạch ba tầng thông tin: dữ liệu cốt lõi của POI, dữ liệu ngữ cảnh chỉ được nạp thêm khi tính score, và nội dung diễn giải được sinh ra ở bước cuối để phục vụ người dùng.

Kế đến, ta trừu tượng hóa khái niệm *mức độ phù hợp tại thời điểm hiện tại* thành một **score** tổng hợp. Dù trên thực tế khái niệm “nên đi đâu bây giờ” là một nhận định mang tính định tính, nhưng để thuật toán có thể xử lý, ta phải biến nó thành một đại lượng có thể tính toán từ các thành phần như khoảng cách, thời tiết, giao thông, mật độ đông đúc và ưu tiên cá nhân. Trong tài liệu đặc tả, lớp trừu tượng này đã được đẩy thêm một bước thành weighted score với các yếu tố cultural value, distance, weather, traffic và estimated crowdedness cùng tham gia vào quyết định cuối cùng [8]. Đây là bước trừu tượng hóa then chốt, vì nó buộc nhóm phải lượng hóa một câu hỏi rất đời thường. Trước khi có lớp abstraction này, mọi trao đổi kiểu “địa điểm này có vẻ hợp hơn” vẫn còn mang nặng trực giác. Khi đã có score, nhóm mới có cơ sở để thảo luận về trọng số, về fallback và về explainability theo đúng nghĩa kỹ thuật.

Tiếp đó, ta trừu tượng hóa *hành trình khám phá văn hóa* thành chuỗi các **Task** có thứ tự, điều kiện mở khóa và tiêu chí hoàn thành. Điều này cực kỳ quan trọng, vì nếu không có lớp trừu tượng này thì storyline sẽ chỉ còn là một đoạn văn giới thiệu dài chứ không thể trở thành một cơ chế tương tác có thể điều khiển được bằng hệ thống. Khi đưa vào thiết kế phần mềm, lớp trừu tượng này xuất hiện dưới dạng kho nhiệm vụ (question pool), lộ trình Mission Path và các nút nhiệm vụ có trạng thái mở khóa trên cả web lẫn mobile. Từ góc nhìn phân tích, đây là bước biến một hành trình mang màu sắc kể chuyện thành một cấu trúc có thể lưu trạng thái, xác minh tiến độ và mở khóa theo điều kiện rõ ràng.

Cuối cùng, ta trừu tượng hóa hành động ngoài đời thật của người dùng thành các **CaptureMetadata** và trạng thái xác minh. Nói cách khác, việc “đã thực sự tới nơi và chụp đúng đối tượng yêu cầu hay chưa” phải được ánh xạ thành dữ liệu đầu vào cho hệ thống kiểm tra. Từ đó mà các agent hoặc service ở phía sau mới có thể đánh giá và phản hồi một cách nhất quán. Nếu không làm bước này, toàn bộ ý tưởng geocaching văn hóa sẽ chỉ còn là tuyên bố ở mức ý tưởng, bởi hệ thống sẽ không có cách nào để nối hành động thật của người dùng với state số của task.

Điểm cốt lõi của abstraction trong BeVietnam là chuyển những khái niệm rất giàu ngữ nghĩa như “một nơi đáng đi”, “một hành trình khám phá” hay “đã thực sự hoàn thành nhiệm vụ” thành các thực thể, trạng thái và đại lượng mà hệ thống có thể xử lý được. Nếu không làm tốt bước này thì mọi phần còn lại, từ scoring đến storyline hay capture verification, đều sẽ chỉ dừng ở mức ý tưởng.

6 System and Algorithm Design

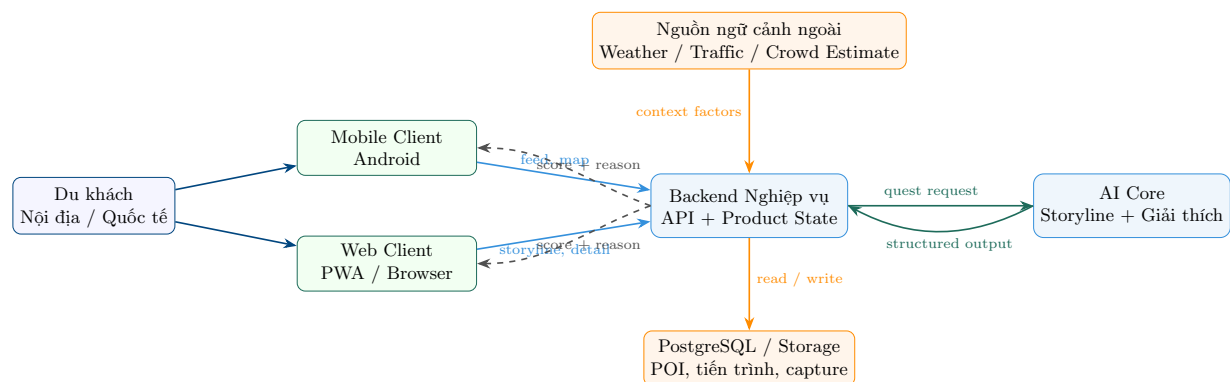


Figure 9: Sơ đồ mức cao của hệ thống BeVietnam. Du khách tương tác qua mobile hoặc web, backend giữ quyền kiểm soát nghiệp vụ và product state, AI Core chỉ xử lý các tác vụ suy luận có cấu trúc, còn dữ liệu ngữ cảnh ngoài được đưa vào để phục vụ recommendation score và storyline.

6.1 Algorithm Design

Sau khi đã phân rã bài toán, nhận diện quy luật và xây dựng mô hình trừu tượng, bước tiếp theo là thiết kế **algorithm**. Trong bối cảnh của BeVietnam, thuật toán không chỉ là một công thức toán học đơn lẻ, mà nó là toàn bộ chuỗi bước xử lý mà hệ thống phải thực hiện để biến dữ liệu thô thành hành động hữu ích cho người dùng [6].

Đối với bài toán feed và bubble map, ý tưởng cốt lõi của thuật toán là: thu thập trạng thái hiện tại của người dùng và môi trường, chuẩn hóa các đặc trưng cần thiết, tính ra điểm phù hợp cho từng địa điểm, sau đó sắp xếp và trình bày kết quả. Nếu viết dưới dạng tư duy thuật toán, tiến trình này có thể được mô tả tường minh như Bảng 11.

Table 11: Quy trình thuật toán cho feed và bubble map

Bước	Thao tác xử lý	Ý nghĩa trong hệ thống
B1	Nhận vị trí hiện tại và tùy chọn sở thích của người dùng.	Thiết lập trạng thái đầu vào ban đầu cho bài toán ra quyết định.
B2	Truy xuất danh sách các địa điểm khả dụng trong khu vực.	Xác định không gian lựa chọn mà hệ thống được phép xếp hạng.
B3	Thu thập thêm thông tin ngữ cảnh như thời tiết, khoảng cách, giao thông hoặc trạng thái hệ thống.	Bổ sung các biến bối cảnh để việc scoring phản ánh đúng thời điểm hiện tại.
B4	Tính recommendation score cho từng địa điểm dựa trên các trọng số đã định nghĩa.	Biến nhận định định tính “nên đi đâu” thành đại lượng có thể tính toán được.
B5	Sắp xếp danh sách theo score và sinh ra phần giải thích ngắn gọn cho từng đề xuất.	Đảm bảo người dùng không chỉ nhận kết quả xếp hạng mà còn hiểu lý do của kết quả đó.
B6	Trả kết quả đã xếp hạng cho feed cá nhân hóa. Riêng bubble map dùng một biến thể thời gian thực: truy vấn POI quanh vị trí GPS và co giãn bong bóng theo khoảng cách thực tế.	Cùng một tư duy ra quyết định theo ngữ cảnh, thể hiện qua hai bề mặt phù hợp với từng mục đích.

Trong quá trình phân tích, nhóm cũng dùng kỹ thuật **if-then rule design** để xác định fallback cho các tình huống lỗi. Chẳng hạn, nếu API thời tiết không phản hồi thì hệ thống không được dừng feed, mà phải dùng giá trị fallback; nếu traffic service lỗi thì ranking vẫn phải chạy với các yếu tố còn lại; nếu AI sinh nhiệm vụ thất bại thì phải trả về quest fallback có cấu trúc thay vì trả về một đoạn văn tự do. Đây là biểu hiện rất rõ của tư duy thuật toán trong bối cảnh hệ thống thật, bởi lời giải không chỉ xử lý ca lý tưởng mà còn phải mô tả rõ ứng xử khi đầu vào thiếu hoặc dịch vụ ngoài gặp sự cố [8]. Cũng chính ở bước này, nhóm mới tách được đâu là phần phải deterministic và đâu là phần có thể để AI hỗ trợ. Những gì liên quan tới ràng buộc nghiệp vụ, quyền truy cập, trạng thái task hay dữ liệu bền vững đều phải nằm ở lớp thuật toán kiểm soát được. Những gì liên quan tới diễn giải, kể chuyện hay đánh giá ngữ nghĩa ảnh mới là phần có thể giao thêm cho AI.

Trong phạm vi pilot, nhóm chưa xử lý đầy đủ dữ liệu giao thông thời gian thực. Vì vậy, yếu tố traffic hiện được mô hình hóa ở mức tham số giả lập hoặc dùng giá trị fallback khi API không khả dụng; việc tích hợp giao thông thật là một hướng nhóm để lại cho giai đoạn sau.

6.2 Ranking Algorithm

Ở phần này nhóm đi sâu vào *cách thực sự suy nghĩ* khi thiết kế lời xếp hạng. Câu hỏi đặt ra là: với một tập POI hoặc nhiệm vụ và một bối cảnh hiện tại, làm sao biến nhận định “cái nào phù hợp hơn” thành một con số? Nhóm đã cân nhắc hai hướng. Hướng thứ nhất là dùng một mô hình học máy để học thứ hạng từ dữ liệu. Hướng thứ hai là dùng một mô hình **cộng trọng số có luật** (weighted additive scoring). Nhóm chọn hướng thứ hai vì ba lý do gắn chặt với bối cảnh pilot, tóm tắt trong Bảng 12.

Table 12: Vì sao chọn weighted additive scoring thay vì học máy

Tiêu chí	Lập luận
Dữ liệu	Pilot chưa có log hành vi đủ lớn để huấn luyện một mô hình xếp hạng đáng tin; cold-start sẽ làm mô hình học máy hoạt động kém.
Giải thích được	Mỗi số hạng cộng vào điểm chính là một <i>lý do</i> có thể hiển thị cho người dùng (“gần bạn”, “hợp thời tiết”), đúng yêu cầu explainable ranking.
Chi phí thời gian thực	Công thức cộng chạy trong vài mili-giây, không cần gọi mô hình tại thời điểm request, phù hợp cho feed và bản đồ tương tác.

6.2.1 Personally Ranking Feed

Xét một bài viết / địa điểm p , tập sở thích của người dùng I , tập nhãn danh mục của bài viết C_p , bối cảnh gồm thời tiết w , buổi trong ngày t , mùa s , và tập địa điểm người dùng đã ghé V . Nhóm định nghĩa điểm phù hợp như một tổng có trọng số tương minh, phát biểu trong Định nghĩa 6.1.

Định nghĩa 6.1: Hàm Điểm Phù Hợp Của Feed

Điểm thô của bài viết p trong bối cảnh (w, t, s) là

$$\text{score}(p) = b + (\alpha + \beta |I \cap C_p|) \mathbf{1}[I \cap C_p \neq \emptyset] + f_w(w, p) + f_t(t, p) + f_s(s, p) - \gamma \mathbf{1}[\text{place}_p \in V],$$

trong đó các hàm khớp ngữ cảnh là hàm bậc thang

$$f_w = \begin{cases} 20 & w \in W_p \\ 6 & \text{any} \in W_p \\ 0 & \text{khác} \end{cases} \quad f_t = \begin{cases} 12 & t \in T_p \\ 4 & \text{any} \in T_p \\ 0 & \text{khác} \end{cases} \quad f_s = \begin{cases} 8 & s \in S_p \\ 0 & \text{khác} \end{cases}$$

và điểm chuẩn hóa về đoạn $[0, 1]$ qua một trần N là $\text{score}(p) = \text{clip}(\text{score}(p)/N, 0, 1)$.

Các tham số được nhóm chốt là $b = 5$ (điểm sàn để mọi bài đều có cơ hội xuất hiện), $\alpha = 10$, $\beta = 5$ (sở thích là tín hiệu cá nhân hóa mạnh nhất), $\gamma = 25$ (phạt novelty để hạ ưu tiên nơi đã đi) và $N = 60$. Thuật toán 1 trình bày toàn bộ vòng xếp hạng. Điểm mấu chốt là cùng tập số hạng vừa cho ra điểm để sắp xếp, vừa cho ra danh sách lý do để giải thích.

Thuật toán 1 Xếp hạng feed theo ngữ cảnh (weighted additive)

Đầu vào: Danh sách $posts$, sở thích I , tập đã ghé V , bối cảnh (w, t, s)

```
1: for all bài viết  $p \in posts$  do
2:    $sc \leftarrow 5$  ▷ điểm sàn
3:    $O \leftarrow I \cap \text{categories}(p)$ 
4:   if  $|O| > 0$  then
5:      $sc \leftarrow sc + 10 + 5|O|$  ▷ tín hiệu sở thích
6:   end if
7:    $sc \leftarrow sc + f_w(w, p) + f_t(t, p) + f_s(s, p)$ 
8:   if  $\text{place}_p \in V$  then
9:      $sc \leftarrow sc - 25$  ▷ phạt novelty
10:  end if
11:   $p.\text{score} \leftarrow \text{clip}(sc/60, 0, 1)$ 
12: end for
13: sắp xếp  $posts$  theo  $score$  giảm dần
14: return  $posts$ 
```

Nhận xét 6.1: Điểm số chính là lời giải thích

Vì điểm là một tổng tường minh, ta có thể truy ngược mỗi địa điểm đã được cộng/trừ ở đâu. Ví dụ một bài viết khớp 2 sở thích, đúng thời tiết nắng, đúng buổi sáng và người dùng chưa từng ghé sẽ có $\text{score} = 5 + (10 + 5 \cdot 2) + 20 + 12 + 0 - 0 = 57$, tức $\text{score} = 57/60 \approx 0.95$. Hệ thống không chỉ trả về 0.95 mà còn trả kèm các lý do “hợp sở thích”, “hợp thời tiết”, “hợp buổi sáng” — đúng tinh thần explainable ranking.

6.2.2 Grading for Mission Path

Khi cần chọn nhiệm vụ phù hợp nhất quanh người dùng từ kho question pool, nhóm dùng cùng triết lý cộng trọng số nhưng thêm **thành phần khoảng cách**. Với nhiệm vụ q có tọa độ và bán kính hiệu lực R_q , gọi d là khoảng cách từ người dùng tới q . Số hạng khoảng cách thưởng mạnh khi người dùng nằm trong bán kính và giảm tuyến tính khi ra xa:

$$g_d(q) = \begin{cases} 40 & d \leq R_q \\ \max(0, 25 - \frac{d - R_q}{200}) & d > R_q \end{cases} \quad (1)$$

Các số hạng còn lại tương tự feed: khớp thời tiết (+20 / +8), phạt -25 nếu trời mưa mà nhiệm vụ ngoài trời, thưởng +12 nếu mưa mà nhiệm vụ trong nhà, khớp buổi (+12 / +4), khớp sở thích ($10 + 5 \cdot$ số nhãn trùng), và +3 cho nhiệm vụ dễ. Thuật toán 2 mô tả logic này, đồng thời thu thập một danh sách *reasons* để giải thích vì sao nhiệm vụ được chọn.

Thuật toán 2 Chấm điểm chọn nhiệm vụ theo ngữ cảnh

Đầu vào: Nhiệm vụ q , bối cảnh (gps, w, t, I)

```
1:  $sc \leftarrow 0$ ;  $reasons \leftarrow \emptyset$ 
2:  $d \leftarrow \text{haversine}(gps, q.coord)$ 
3: if  $d \leq R_q$  then
4:    $sc \leftarrow sc + 40$ ; thêm near_user vào  $reasons$ 
5: else
6:    $sc \leftarrow sc + \max(0, 25 - (d - R_q)/200)$ 
7: end if
8:  $sc \leftarrow sc + f_w(w, q) + f_t(t, q)$  ▷ khớp thời tiết / buổi
9: if  $w = \text{rainy}$  và  $q$  ngoài trời then
10:    $sc \leftarrow sc - 25$ 
11: end if
12: if  $w = \text{rainy}$  và  $q$  trong nhà then
13:    $sc \leftarrow sc + 12$ 
14: end if
15:  $O \leftarrow I \cap q.categories$ 
16: if  $|O| > 0$  then
17:    $sc \leftarrow sc + 10 + 5|O|$ ; thêm interest_match
18: end if
19: if  $q.difficulty = \text{easy}$  then
20:    $sc \leftarrow sc + 3$ 
21: end if
22: return  $(sc, reasons)$ 
```

6.2.3 Dynamic Bubble Marker

Bản đồ thời gian thực không xếp hạng theo công thức trên mà chấm theo khoảng cách thuần túy. Khoảng cách trắc địa giữa người dùng và POI được tính bằng công thức **haversine** [10]:

$$d = 2R \arcsin \sqrt{\sin^2 \frac{\Delta\phi}{2} + \cos \phi_1 \cos \phi_2 \sin^2 \frac{\Delta\lambda}{2}} \quad (2)$$

với ϕ là vĩ độ, λ là kinh độ và R là bán kính Trái Đất. Sau đó bán kính bong bóng được nội suy tuyến tính theo trọng số $\omega = 1 - d/d_{\max}$ như đã trình bày ở Pseudocode 1, để địa điểm càng gần thì bong bóng càng lớn.

6.3 Thuật Toán Điều Hướng và Xác Minh

Đối với storyline, thuật toán lại mang màu sắc của một bài toán điều hướng tiến trình. Từ một điểm đến ban đầu, hệ thống phải xác định nhiệm vụ nào là hợp lý tiếp theo, nhiệm vụ đó yêu cầu bằng chứng gì, điều kiện nào được xem là hoàn thành và khi nào thì mở khóa bước kế tiếp. Điều này có nghĩa là ta đang thiết kế một *state machine* ở mức khái niệm, trong đó mỗi trạng thái tương ứng với một mốc tiến độ của hành trình văn hóa.

Đối với xác minh capture, quy trình giải kết hợp một bước suy luận thị giác với một khung quyết định tường minh. Hệ thống nhận ảnh người dùng, lấy một ảnh tham chiếu của chủ thể nhiệm vụ, rồi yêu cầu mô hình thị giác chấm điểm độ khớp theo một rubric chia thang 0–1. Điểm trả về được ánh xạ qua các ngưỡng cố định thành *approved*, *needs_review* hay *rejected*, và chỉ trạng thái *approved* mới mở khóa nút kế tiếp. Đây là một ví dụ điển hình cho việc biến một yêu cầu đòi thực mang tính định tính (“ảnh này có đúng địa điểm không”) thành một pipeline xử lý vừa tận dụng được sức mạnh suy luận của AI, vừa giữ

được tính kiểm soát qua rubric và ngưỡng.

Điều đặc biệt quan trọng là trong BeVietnam, nhiều thuật toán không nên được đặt hoàn toàn bên trong mô hình AI. AI chỉ nên tham gia ở những khâu cần sinh ngôn ngữ, diễn giải văn hóa hoặc đánh giá ảnh ở mức ngữ nghĩa. Còn cấu trúc điều hướng nghiệp vụ, cập nhật trạng thái, ràng buộc API và kiểm tra tính hợp lệ của dữ liệu vẫn phải nằm trong backend và schema có cấu trúc. Đây chính là biểu hiện của tư duy tính toán đúng đắn: thuật toán phải minh bạch, kiểm soát được và không phụ thuộc hoàn toàn vào một hộp đen sinh ngẫu nhiên. Chính vì vậy, trong giai đoạn phân tích, nhóm ưu tiên thiết kế contract API, schema output và ràng buộc state transition trước khi bàn sâu đến việc chọn model nào mạnh hơn.

6.4 Evaluation

Bước cuối cùng của Computational Thinking là **evaluation**, tức là đánh giá xem lời giải mà ta xây dựng có thực sự giải quyết bài toán ban đầu hay không [6]. Với một sản phẩm như BeVietnam, đây là bước không thể thiếu, bởi một thiết kế nghe có vẻ hợp lý trên giấy chưa chắc đã hữu ích khi đưa vào bối cảnh du lịch thực tế.

Trong BeVietnam, evaluation không phải là công việc để cuối cùng mới làm, mà là bộ tiêu chuẩn được dùng để phản biện lại toàn bộ lời giải ngay từ giai đoạn phân tích. Mỗi khi đề xuất một tính năng hay một pipeline, nhóm đều đặt câu hỏi ngược lại rằng ta sẽ biết nó đúng bằng cách nào, đo được bằng gì và nếu nó thất bại thì thất bại ở đâu. Cách đặt vấn đề này giúp nhóm tránh được tình huống xây xong hệ thống rồi mới phát hiện ra rằng không có tiêu chí rõ ràng để kiểm tra.

Lớp đánh giá đầu tiên là **đúng chức năng**. Ở đây, nhóm đối chiếu trực tiếp với các functional requirement của pilot Huế: feed phải trả về giải thích điểm số, bubble map phải dùng cùng một score như feed, storyline phải mở được nhiệm vụ kế tiếp, và capture phải lưu được metadata một cách nhất quán [8]. Điểm quan trọng là nhóm không xem việc API trả về dữ liệu là đủ, mà phải kiểm tra xem mỗi thành phần có thực sự hoàn thành đúng vai trò của nó trong vòng lặp sản phẩm hay chưa.

Lớp đánh giá thứ hai là **chất lượng trải nghiệm**. Một kết quả có thể đúng về mặt dữ liệu nhưng vẫn không hữu ích nếu lời giải thích quá chung chung, nhiệm vụ quá khó hiểu hoặc luồng xác minh gây phiền hà. Vì vậy, evaluation trong BeVietnam còn phải đi qua góc nhìn của người dùng thật: họ có hiểu vì sao một địa điểm được đề xuất hay không, họ có thấy nhiệm vụ đủ hấp dẫn để tiếp tục hành trình hay không. Chính từ logic này mà phần đặc tả pilot đã đưa luôn các success metric như tỷ lệ người dùng thấy gợi ý hữu ích, tỷ lệ học thêm được nội dung văn hóa mới và tỷ lệ hoàn thành ít nhất một nhiệm vụ storyline [8]. Nói cách khác, hệ thống không chỉ được đo bằng log kỹ thuật, mà còn bằng khả năng tạo ra giá trị du lịch và giá trị học hỏi văn hóa cho người dùng.

Lớp đánh giá thứ ba là **độ tin cậy của AI**. Các agent văn hóa hay task generation không thể chỉ được xem là “chạy ra kết quả” là đủ, mà còn phải được kiểm tra về tính đúng đắn của nội dung, tính an toàn, cấu trúc đầu ra và khả năng fallback khi mô hình trả về kết quả kém chất lượng. Đây là lý do tài liệu kỹ thuật của BeVietnam đặt ra các nguyên tắc *structured output*, *retrieval before generation* và *fail safe* [8]. Cũng từ góc nhìn evaluation mà nhóm không chọn cách làm chatbot mở hoàn toàn, bởi loại hệ thống đó rất khó kiểm tra đúng sai một cách nhất quán. Ngược lại, các output có schema, có fallback và có dữ liệu nền rõ ràng thì dễ kiểm chứng hơn rất nhiều.

Lớp đánh giá cuối cùng là **khả năng mở rộng**. Một lời giải tốt không chỉ giải được Huế trong giai đoạn pilot, mà còn phải giữ được cấu trúc đủ sạch để có thể mở rộng sang những địa phương khác về sau. Nếu các mô hình dữ liệu, API và pipeline xử lý được thiết kế đúng từ đầu thì việc bổ sung địa điểm, câu

chuyện văn hóa hay nguồn dữ liệu mới sẽ dễ dàng hơn rất nhiều. Cũng chính vì nhìn bài toán dưới lăng kính này mà nhóm chốt thêm một quyết định quan trọng: pilot Huế phải được đóng phạm vi chặt. Nếu mở rộng quá sớm ra nhiều địa phương, nhóm sẽ không thể đánh giá công bằng chất lượng dữ liệu, chất lượng storyline và độ đúng của recommendation score.

Do đó ta có thể thấy rằng, evaluation trong BeVietnam không chỉ là bước “kiểm thử xem code có chạy hay không”, mà là quá trình phản biện liên tục đối với cả kỹ thuật, sản phẩm lẫn trải nghiệm thực tế. Chính nhờ lớp đánh giá này mà tư duy tính toán mới trở thành một vòng lặp khép kín: ta phân tích bài toán, thiết kế lời giải, triển khai hệ thống, quan sát kết quả, rồi quay lại tinh chỉnh mô hình và thuật toán khi cần thiết.

Evaluation trong BeVietnam không đứng tách biệt ở cuối dự án. Nó xuất hiện ngay từ lúc nhóm chọn scope pilot, xác định functional requirement, ràng buộc output của AI và đặt ra success metric cho người dùng. Chính nhờ đó mà toàn bộ lời giải của hệ thống mới giữ được tính kiểm chứng, thay vì trôi dần về những tính năng khó đo lường hoặc khó vận hành.

7 Triển khai Hệ thống

Sau khi đã phân tích bài toán và thiết kế lời giải qua các chương trước, chương này trình bày cách BeVietnam được **triển khai** thành một hệ thống chạy được. Mục tiêu không phải liệt kê toàn bộ mã nguồn, mà là làm rõ kiến trúc tổng thể, vai trò của từng dịch vụ và cách ba tính năng trọng tâm — bản đồ thời gian thực, Mission Path và xác minh ảnh bằng thị giác — được nối với nhau qua các thuật toán cụ thể.

7.1 Kiến Trúc Tổng Thể

Hệ thống được tổ chức thành một *monorepo* gồm năm thành phần triển khai độc lập nhưng phối hợp chặt với nhau. Bảng 13 tóm tắt vai trò và công nghệ của từng thành phần, còn Hình 10 mô tả luồng dữ liệu giữa chúng.

Table 13: Các thành phần triển khai của BeVietnam

Thành phần	Công nghệ	Vai trò chính
Web client	Next.js (App Router), TypeScript	Feed, Explore (bubble map), Mission Path storyline, capture qua trình duyệt.
Mobile client	Android, Jetpack Compose	Trải nghiệm bản đồ thời gian thực, định vị GPS, chụp ảnh thực địa.
Backend	FastAPI (async), PostgreSQL, MinIO	Giữ product state, proxy dịch vụ ngoài, xác thực JWT, điều phối xác minh capture.
AI Core	FastAPI, các agent, langgraph	Sinh question pool (offline), giải thích đề xuất, và Capture Judge thị giác (runtime).
vLLM hosting	vLLM, Qwen2.5-VL-7B, 1× GPU L40	Phục vụ mô hình thị giác cho việc chấm điểm ảnh minh chứng.

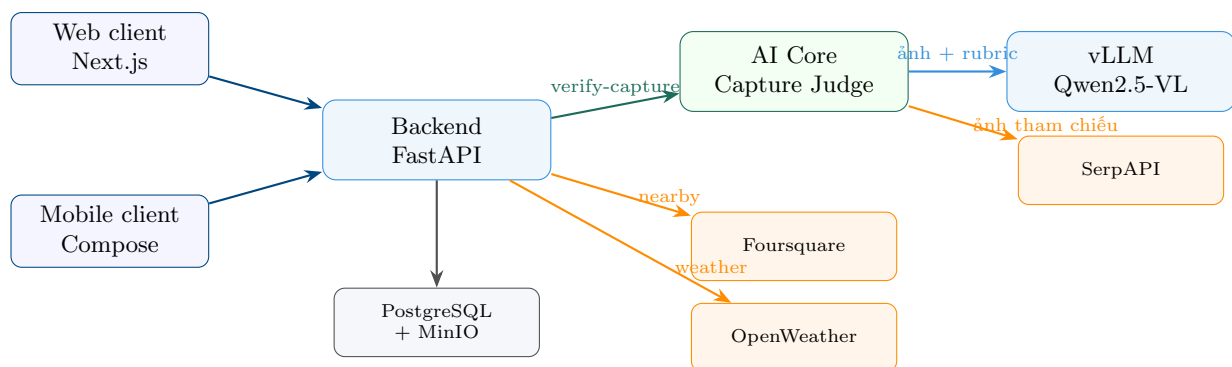


Figure 10: Kiến trúc triển khai BeVietnam. Hai client gọi vào backend; backend giữ product state (PostgreSQL/MinIO) và proxy các dịch vụ ngoài (Foursquare, OpenWeather). Riêng việc xác minh ảnh được backend chuyển tiếp cho AI Core, nơi mô hình thị giác Qwen2.5-VL (tự host trên vLLM) thực hiện chấm điểm; ảnh tham chiếu được lấy qua SerpAPI và lưu cache.

Một nguyên tắc xuyên suốt là **backend giữ quyền kiểm soát**, còn các dịch vụ ngoài và AI chỉ là nguồn dữ liệu hoặc bộ suy luận được gọi khi cần. API của các dịch vụ trả phí (Foursquare, OpenWeather, Goong) đều được giữ ở server và đi qua một lớp *rate limiter* chung để bảo vệ hạn mức free-tier trong giai đoạn demo.

7.2 Kiến Trúc theo Mô Hình C4

Để mô tả kiến trúc một cách có hệ thống, nhóm dùng **mô hình C4**, trong đó hệ thống được nhìn ở nhiều mức trừu tượng khác nhau: từ mức *bối cảnh* trừu tượng nhất cho tới mức *thành phần* chi tiết bên trong từng phía. Phần này trình bày bốn lớp: một lớp trừu tượng tổng quan, rồi ba lớp chi tiết lần lượt cho frontend, backend và AI service.

7.2.1 Abstract Layer

Ở mức trừu tượng nhất (Hình 11), toàn bộ hệ thống được xem như một hộp duy nhất: du khách tương tác với BeVietnam, còn BeVietnam gọi tới một số dịch vụ ngoài để lấy POI, thời tiết, ảnh tham chiếu và bản đồ nền.

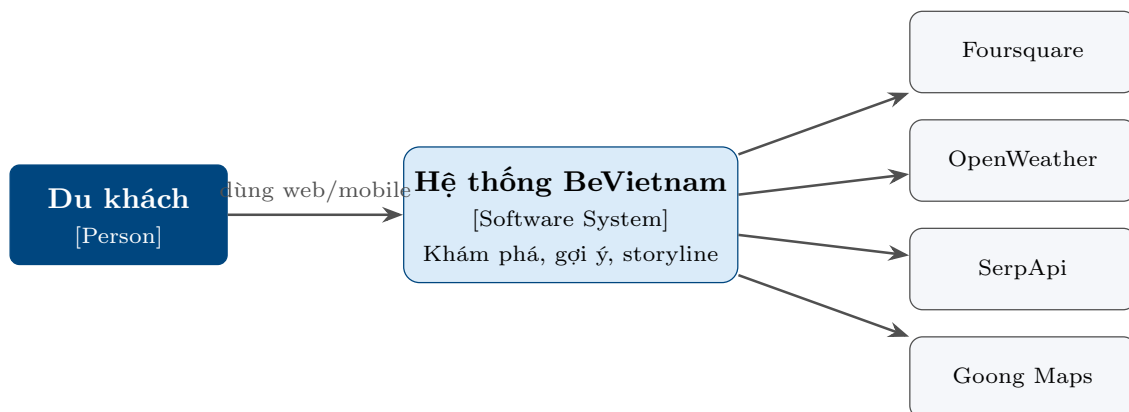


Figure 11: Du khách dùng BeVietnam; hệ thống lấy dữ liệu từ bốn dịch vụ ngoài.

7.2.2 Frontend Layer

Mở hộp “client” ra, ta có hai container giao diện dùng chung một backend (Hình 12). Web và mobile có cấu trúc tương tự: một lớp màn hình/tính năng, một lớp gọi API, và (ở mobile) một lớp ViewModel + repository theo kiến trúc luồng dữ liệu một chiều.

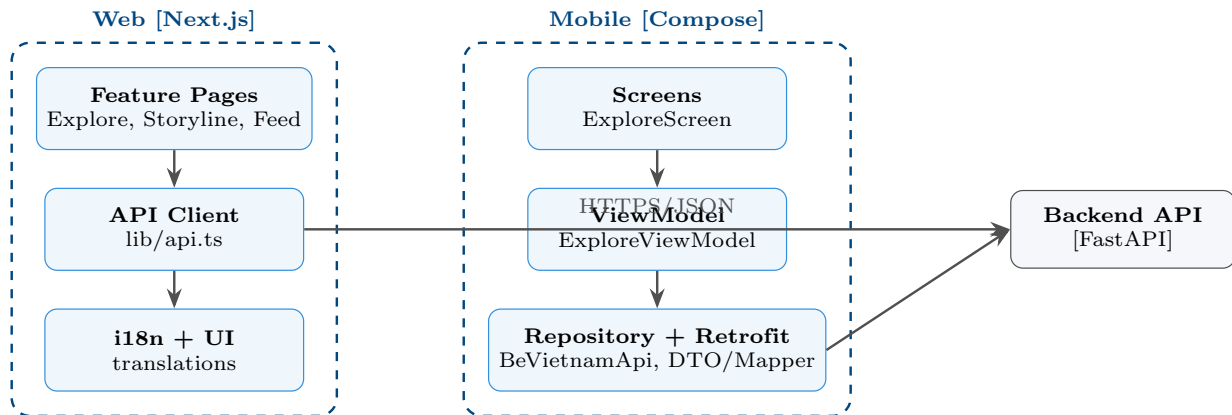


Figure 12: C4 mức container/thành phần — phía frontend. Web (Next.js) và mobile (Compose) chia sẻ cùng một backend qua HTTPS/JSON.

7.2.3 Backend Layer

Bên trong container backend (Hình 13), các thành phần được xếp theo tầng: lớp *endpoint* nhận request, gọi xuống lớp *service* (ng nghiệp vụ và proxy dịch vụ ngoài) và lớp *repository* (đọc/ghi dữ liệu). Các thành phần xuyên suốt như rate limiter, xác thực JWT và AI Core Client phục vụ cho nhiều endpoint.

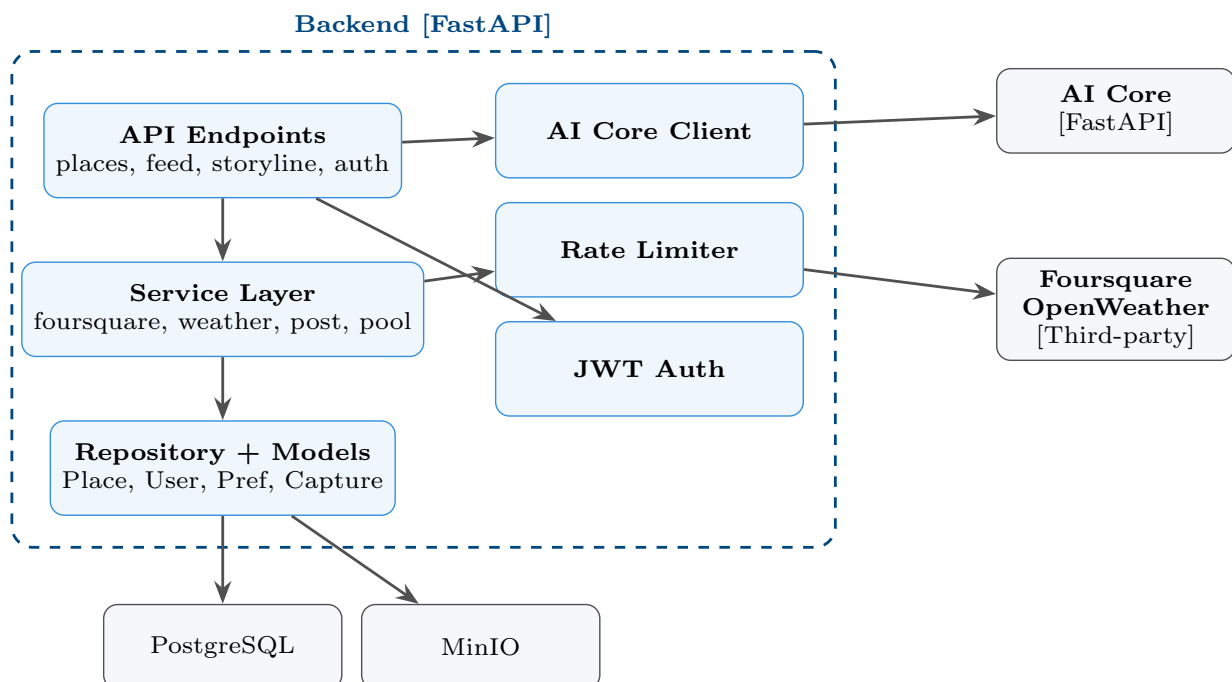


Figure 13: C4 mức thành phần — phía backend. Endpoint → service → repository; rate limiter chặn các lời gọi dịch vụ ngoài; AI Core Client chuyển tiếp việc xác minh ảnh.

7.2.4 AI Service

Cuối cùng, mở container AI Core (Hình 14). Phần nội dung (question pool, bài viết) do các agent sinh offline, còn ở runtime chỉ agent **Capture Judge** hoạt động: nó lấy ảnh tham chiếu rồi gọi mô hình thị giác qua một LLM Gateway thống nhất.

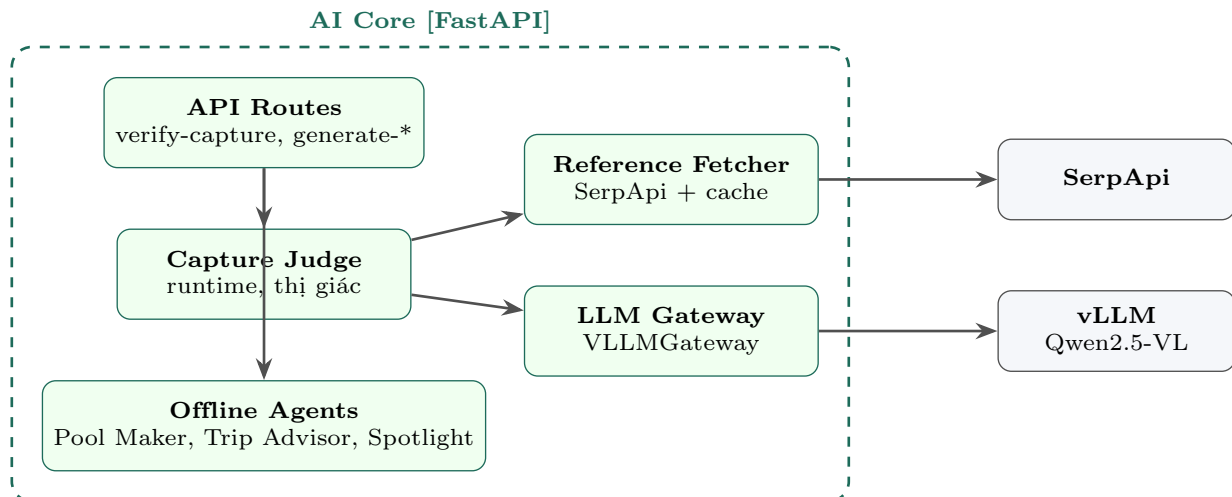


Figure 14: C4 mức thành phần — AI service. Capture Judge là agent runtime duy nhất, dùng Reference Fetcher (SerpApi + cache) và LLM Gateway tới mô hình thị giác vLLM.

7.3 Lớp Dữ Liệu và API

Bốn thực thể trừu tượng ở Chương Abstraction được hiện thực thành các bảng và endpoint cụ thể (Bảng 14). Backend dùng PostgreSQL cho dữ liệu có cấu trúc (POI, người dùng, tùy chọn cá nhân, capture) và MinIO cho ảnh.

Table 14: Một số endpoint chính của backend

Endpoint	Vai trò
GET /places/nearby	POI thời gian thực quanh một tọa độ (proxy Foursquare).
GET /weather	Thời tiết khu vực: nhiệt độ, UV ước lượng, lượng mưa.
GET /question-pool	Toàn bộ kho nhiệm vụ cho Mission Path.
GET /feed	Feed gợi ý đã cá nhân hóa theo tùy chọn người dùng.
POST /storyline/verify-capture	Chuyển ảnh minh chứng sang AI Core để chấm điểm.
GET/PUT /me/preferences	Đọc/ghi tùy chọn cá nhân phục vụ xếp hạng feed.

7.4 Realtime Bubble Map

Khi khảo sát các ứng dụng bản đồ và review, nhóm nhận thấy người dùng phải mở nhiều nguồn khác nhau để xem địa điểm, thời tiết và mô tả. Vì vậy, trong bản pilot tại Huế, nhóm chọn cách gom các thông tin cần thiết về một endpoint để client chỉ cần gọi một lần khi mở màn hình Explore.

Bản đồ thời gian thực neo vào vị trí GPS thực của người dùng. Khi vào màn hình, client xin quyền và lấy tọa độ hiện tại, sau đó gọi /places/nearby với bán kính cố định 3km, được backend lấy từ Foursquare Places API [11]. Mỗi POI trả về được client vẽ thành một bong bóng, với bán kính co giãn theo khoảng

cách *haversine* [10] tới người dùng: càng gần thì bong bóng càng lớn. Pseudocode 1 mô tả bước dựng bong bóng.

Listing 1: Dựng bong bóng theo khoảng cách tới người dùng

```
1 INPUT places[], user_gps
2 maxDist = max( haversine(user_gps, p.coord) for p in places )
3 FOR each p in places:
4     d = haversine(user_gps, p.coord)
5     weight = 1 - d / maxDist # 1 = gần nhất, 0 = xa nhất
6     radius = BASE + weight * SPREAD # px
7     color = colorOfCategory(p.bucket) # food/lodging/culture/...
8     emit bubble(center = p.coord, radius, color, label = p.name)
```

Toàn bộ POI là dữ liệu sống của Foursquare nên không cố định theo địa phương; ứng dụng hoạt động ở bất kỳ đâu người dùng đứng. Một chip thời tiết của khu vực (nhiệt độ, UV, mưa) lấy từ OpenWeather API [12] được hiển thị kèm để bổ sung ngữ cảnh ra quyết định.

7.5 Storyline Mission Path

Mission Path nạp toàn bộ kho nhiệm vụ từ /question-pool rồi dựng một lộ trình dạng các nút uốn lượn. Trạng thái của mỗi nút được suy ra hoàn toàn từ tập nhiệm vụ đã hoàn thành (lưu cục bộ), theo luật mở khóa tuần tự ở Pseudocode 2.

Listing 2: Luật trạng thái nút trên Mission Path

```
1 INPUT tasks[], completedIds
2 activeIndex = first index i where tasks[i].id not in completedIds
3 FOR each task[i]:
4     IF task[i].id in completedIds: state = DONE # đã hoàn thành
5     ELIF i == activeIndex: state = ACTIVE # đang mở, có thể làm
6     ELSE: state = LOCKED # chưa mở khóa
```

Khi người dùng chạm vào một nút đang mở, hệ thống hiển thị chi tiết nhiệm vụ (tiêu đề, yêu cầu, giải nghĩa văn hóa) cùng nút chụp/tải ảnh. Ảnh được gửi đi xác minh; nếu đạt, nhiệm vụ chuyển sang DONE và nút kế tiếp tự động trở thành ACTIVE.

7.6 Vision Capture Verification

Đây là tính năng **duy nhất** bắt buộc dùng mô hình AI tại thời gian thực. Khi backend nhận ảnh và chuyển sang AI Core, agent **Capture Judge** thực hiện một pipeline bốn bước (Hình 15): giải mã ảnh người dùng, lấy ảnh tham chiếu của chủ thể nhiệm vụ (tìm qua SerpApi Google Images [13] rồi lưu cache theo từng nhiệm vụ), yêu cầu mô hình thị giác chấm điểm theo rubric, và ánh xạ điểm sang trạng thái cuối cùng.

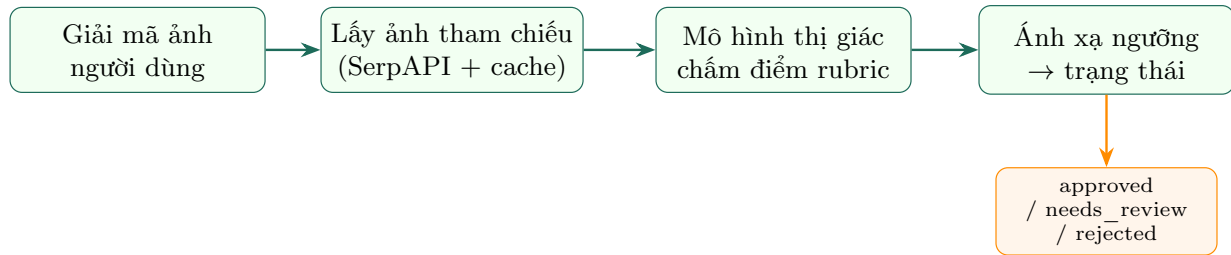


Figure 15: Pipeline xác minh ảnh của Capture Judge. Nếu không tìm được ảnh tham chiếu, mô hình vẫn chấm dựa trên mô tả nhiệm vụ; nếu mô hình không sẵn sàng, hệ thống trả *needs_review* thay vì tự ý chấp nhận.

Rubric chấm điểm được đưa thẳng vào system prompt của mô hình để kết quả mang tính tất định hơn (Bảng 15). Mô hình nhận hai ảnh (tham chiếu và của người dùng) cùng mô tả nhiệm vụ, rồi trả về một JSON gồm điểm, nhãn và lý do.

Table 15: Rubric chấm điểm độ khớp ảnh (thang 0–1)

Khoảng điểm	Diễn giải
0.0 – 0.2	Hoàn toàn khác, không liên quan đến chủ thể yêu cầu.
0.2 – 0.4	Vẫn khác, chỉ nhỉnh hơn một chút (đúng chủ đề chung nhưng sai đối tượng).
0.4 – 0.6	Gần đúng nhưng vẫn sai địa điểm (ví dụ: yêu cầu “Ngọ Môn” nhưng ảnh là “Tứ Cấm Thành”).
0.7 – 0.9	Ảnh tốt, đúng chủ thể/địa điểm được yêu cầu.
1.0	Trùng khớp rõ ràng, không nghi ngờ.

Quy tắc quyết định được phát biểu tường minh trong Định nghĩa 7.1, với hai ngưỡng $\tau_a = 0.7$ và $\tau_r = 0.4$ phân tách ba vùng.

Định nghĩa 7.1: Quy Tắc Quyết Định Xác Minh

Cho điểm khớp $\sigma \in [0, 1]$ do mô hình thị giác trả về (hoặc σ không xác định khi mô hình không sẵn sàng), trạng thái xác minh là

$$\text{status}(\sigma) = \begin{cases} \text{approved} & \sigma \geq \tau_a \\ \text{needs_review} & \tau_r \leq \sigma < \tau_a \text{ hoặc } \sigma \text{ không xác định} \\ \text{rejected} & \sigma < \tau_r \end{cases}$$

Chỉ trạng thái *approved* mới mở khóa nút nhiệm vụ kế tiếp.

Thuật toán 3 trình bày toàn bộ luồng từ ảnh người dùng tới trạng thái cuối.

Thuật toán 3 Ánh xạ điểm rubric sang trạng thái xác minh

Đầu vào: Ảnh người dùng img , nhiệm vụ q

```
1:  $ref \leftarrow \text{getReference}(q.id, \text{subject}(q))$  ▷ cache hoặc SerpAPI  
2:  $\sigma \leftarrow \text{VLM.judge}([ref, img], \text{rubric}, q)$  ▷  $\in [0, 1]$   
3: if  $\sigma$  không xác định then  
4:   return needs_review  
5: end if  
6: if  $\sigma \geq 0.7$  then  
7:   return approved  
8: else if  $\sigma \geq 0.4$  then  
9:   return needs_review  
10: else  
11:   return rejected  
12: end if
```

Nhận xét 7.1: Kiểm soát AI bằng cấu trúc

Điểm đáng chú ý về tư duy tính toán ở đây là cách AI được “đóng khung”. Thay vì hỏi mô hình “ảnh này có hợp lệ không” và tin tuyệt đối vào câu trả lời, hệ thống cung cấp ảnh tham chiếu làm mốc so sánh, ép mô hình chấm theo một rubric tường minh, và để các **ngưỡng số** ở phía backend quyết định trạng thái. Nhờ vậy, một thành phần vốn rất “mềm” như suy luận thị giác vẫn được đưa về một pipeline có đầu vào, đầu ra và quy tắc rõ ràng — đúng tinh thần biến cái định tính thành cái tính toán được.

Phần xác minh ảnh hiện chỉ phù hợp với các nhiệm vụ có chủ thể quan sát rõ (một công trình, một chi tiết kiến trúc cụ thể). Với các nhiệm vụ mang tính cảm nhận văn hóa hoặc hành vi — ví dụ “cảm nhận không khí phiêu chợ” — mô hình thị giác khó chấm chính xác, nên nhóm vẫn cần một cơ chế duyệt thủ công (trạng thái `needs_review`) cho nhóm nhiệm vụ đó.

7.7 Mô Hình AI Tự Host

Trong toàn bộ hệ thống, chỉ có **một** tính năng cần mô hình AI ở thời gian thực là Capture Judge; các nội dung khác (question pool, bài viết feed) được sinh trước offline. Vì việc xác minh ảnh là bài toán thị giác, nhóm chọn một mô hình ngôn ngữ-thị giác (VLM) là **Qwen2.5-VL-7B-Instruct** [14] và tự host trên vLLM [15, 16] với một GPU L40 (48 GB). Một mô hình bản 7B ở định dạng bfloat16 chiếm khoảng 16 GB, còn dư nhiều bộ nhớ cho KV-cache (cơ chế quản lý bộ nhớ PagedAttention của vLLM [15]), nên một card L40 là đủ để phục vụ tính năng này mà không cần thêm phần cứng. Việc dùng mô hình thị giác cũng cho phép gửi đồng thời *hai* ảnh (tham chiếu và của người dùng) trong một lượt suy luận để mô hình so sánh trực tiếp.

8 Kiểm Thử và Đánh Giá

Như đã nêu ở phần Evaluation, nhóm xem việc đánh giá là một vòng phản biện chạy song song với phát triển chứ không phải bước cuối. Chương này trình bày cách kiểm thử thực tế trong giai đoạn pilot

và đối chiếu kết quả với các tiêu chí đã đặt ra.

8.1 Phương Pháp Kiểm Thử

Vì BeVietnam là một hệ thống nhiều thành phần (hai client, backend, AI Core, vLLM), nhóm kiểm thử theo bốn lớp từ thấp đến cao, tóm tắt trong Bảng 16.

Table 16: Bốn lớp kiểm thử của hệ thống

Lớp kiểm thử	Cách thực hiện
Biên dịch / kiểu dữ liệu	Kiểm tra web bằng trình biên dịch TypeScript, mobile bằng Gradle, mã Python của AI bằng <code>py_compile</code> ; đảm bảo toàn bộ build không lỗi trước khi tích hợp.
Tích hợp dịch vụ	Gọi trực tiếp các endpoint qua đường hầm public để xác nhận backend, proxy dịch vụ ngoài và AI Core hoạt động đúng từ ngoài mạng.
Trải nghiệm trên thiết bị	Cài bản APK thực lên điện thoại Android, kiểm tra bản đồ thời gian thực, định vị GPS và luồng chụp ảnh tại hiện trường.
Hành vi AI	Thử nghiệm Capture Judge với ảnh đúng và ảnh sai chủ thể để kiểm tra xem rubric có phân biệt được hay không.

8.2 Kết Quả Kiểm Thử Chính

Bảng 17 liệt kê một số ca kiểm thử tiêu biểu và kết quả quan sát được. Các ca tích hợp được thực hiện bằng cách gọi endpoint qua đường hầm Cloudflare, mô phỏng đúng cách client thật truy cập hệ thống.

Table 17: Một số ca kiểm thử tiêu biểu và kết quả

Ca kiểm thử	Kỳ vọng	Kết quả
POI thời gian thực tại trung tâm TP.HCM	Trả về danh sách quán/địa điểm quanh tọa độ, có phân loại nhóm.	Đạt
Kho nhiệm vụ /question-pool	Trả về toàn bộ nhiệm vụ cho Mission Path.	Đạt (133 nhiệm vụ)
Thời tiết khu vực	Trả về nhiệt độ, UV ước lượng, lượng mưa.	Đạt
Xác minh ảnh <i>sai</i> chủ thể	Không được chấp nhận (rejected / needs_review).	Đạt sau khi thay stub bằng mô hình thị giác
Build web (production)	Biên dịch và tạo trang /storyline thành công.	Đạt
Build mobile (APK debug)	Gradle hoàn tất, tạo được file cài đặt.	Đạt

Một quan sát quan trọng trong quá trình kiểm thử AI: phiên bản đầu của Capture Judge chỉ kiểm tra “có ảnh hay không” rồi chấp nhận ở mức tin cậy cao, nên một ảnh hoàn toàn không liên quan vẫn được duyệt. Đây chính là loại lỗi mà chỉ kiểm thử hành vi mới phát hiện được, và là lý do nhóm thay thế bằng pipeline rubric + ảnh tham chiếu + mô hình thị giác mô tả ở chương trước.

8.3 Đánh Giá theo Bốn Tiêu Chí

Nhóm đối chiếu hệ thống đã hiện thực với bốn lớp tiêu chí đã đặt ra ở chương System and Algorithm Design.

Đúng chức năng. Ba vòng lặp sản phẩm cốt lõi đều chạy thông từ đầu đến cuối: người dùng mở bản đồ và thấy POI thời gian thực quanh mình; mở Mission Path và đi qua các nút nhiệm vụ; chụp ảnh và

nhận kết quả xác minh có cơ sở. Các endpoint nền tảng đều phản hồi đúng khi gọi từ ngoài mạng qua đường hầm.

Chất lượng trải nghiệm. Việc đồng bộ giao diện mobile theo hệ màu Huế của web, bổ sung nhãn tên địa điểm trên bong bóng và nút định vị nhanh giúp trải nghiệm liền mạch hơn giữa hai nền tảng. Mission Path dạng lộ trình trực quan giúp người dùng nắm được tiến độ rõ ràng thay vì một danh sách phẳng.

Độ tin cậy của AI. Sau khi thay stub bằng mô hình thị giác có rubric, hệ thống không còn duyệt bữa ảnh sai chủ thể. Cơ chế *needs_review* bảo đảm rằng khi mô hình không sẵn sàng hoặc còn nghi ngờ, hệ thống không tự ý chấp nhận — một hành vi an toàn quan trọng trong pilot.

Khả năng mở rộng. Việc khóa nguồn POI về một endpoint dùng chung cho cả hai client, tách nội dung nhiệm vụ (sinh offline) khỏi luồng điều hướng, và đặt khóa dịch vụ ngoài sau một rate limiter chung khiến hệ thống dễ nhân rộng sang địa phương khác mà không phải viết lại lõi.

Trong phạm vi pilot, kiểm thử còn nghiêng về kiểm tra tích hợp và hành vi thủ công, chưa có bộ kiểm thử tự động đầy đủ hay đánh giá người dùng quy mô lớn. Độ chính xác của mô hình thị giác cũng phụ thuộc vào chất lượng ảnh tham chiếu lấy từ tìm kiếm web. Đây là những hướng cần củng cố khi đưa hệ thống ra khỏi giai đoạn thí điểm.

9 Triển Khai

Chương này trình bày cách BeVietnam được đưa lên môi trường chạy thật cho giai đoạn pilot, từ cách bố trí dịch vụ trên máy chủ GPU đến cơ chế công khai hệ thống ra Internet và cách bảo vệ các thông tin bí mật.

9.1 Bố Trí Triển Khai

Toàn bộ phần lõi phía server được gom về một máy chủ GPU duy nhất (1 card L40) để đơn giản hóa vận hành trong pilot. Trên máy này, một script khởi tạo dựng lần lượt các dịch vụ: cơ sở dữ liệu, lưu trữ ảnh, mô hình thị giác, AI Core và backend; cuối cùng là một đường hầm Cloudflare để công khai backend ra một tên miền cố định. Web client chạy độc lập và trở về tên miền này, còn mobile được phân phối dưới dạng APK cài trực tiếp lên điện thoại. Hình 16 mô tả bố trí đó.

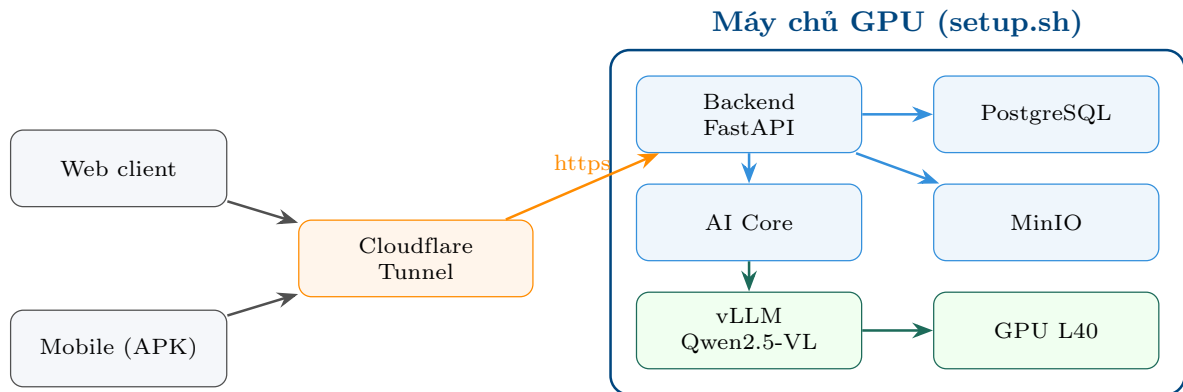


Figure 16: Bố trí triển khai pilot. Toàn bộ dịch vụ server nằm trên một máy chủ GPU, được dựng tự động bởi script `setup.sh`. Cloudflare Tunnel công khai backend ra một tên miền cố định; cả web lẫn mobile đều truy cập qua đường hầm này nên hoạt động được trên mọi mạng.

9.2 Tự Động Hóa Khởi Tạo

Việc đưa nhiều dịch vụ lên cùng một máy được gói trong một script khởi tạo duy nhất, chạy tuần tự theo thứ tự phụ thuộc (Bảng 18). Cách này giúp một lệnh duy nhất có thể dựng lại toàn bộ stack, đồng thời ghi cấu hình ra các file môi trường riêng cho từng dịch vụ.

Table 18: Trình tự khởi tạo của script triển khai

Bước	Dịch vụ	Vai trò
1	PostgreSQL	Khởi tạo cơ sở dữ liệu và chạy migration.
2	MinIO	Dựng kho lưu trữ ảnh capture.
3	vLLM	Tải và phục vụ mô hình thị giác Qwen2.5-VL trên GPU.
4	AI Core	Khởi động dịch vụ agent (gồm Capture Judge).
5	Backend	Khởi động API nghiệp vụ, kết nối DB, MinIO và AI Core.
6	Cloudflare Tunnel	Công khai backend ra tên miền cố định.

Riêng vLLM [16], vì phải phục vụ mô hình thị giác, cần một cấu hình đặc thù: chọn đúng model VLM, cho phép gửi tối đa hai ảnh trong một yêu cầu (ảnh tham chiếu và ảnh người dùng), và đặt độ dài ngữ cảnh đủ lớn vì token ảnh khá nặng. Một card L40 48 GB đủ chỗ cho mô hình bản 7B cùng KV-cache nên không cần thêm GPU.

9.3 Bảo Mật và Cấu Hình Bí Mật

Một nguyên tắc vận hành được tuân thủ chặt là **không đưa thông tin bí mật vào mã nguồn quản lý phiên bản**. Các khóa dịch vụ (Foursquare, OpenWeather, Goong, SerpAPI, token mô hình), bí mật của đường hầm và file cấu hình thực thi đều được giữ ngoài Git. Backend chỉ lắng nghe nội bộ và chỉ được công khai gián tiếp qua đường hầm Cloudflare, nên không mở cổng trực tiếp ra Internet công cộng. Các ảnh tham chiếu tải về từ web phục vụ việc xác minh cũng được lưu cache cục bộ và loại khỏi Git để tránh phát tán nội dung có bản quyền.

Việc gom toàn bộ server lên một máy GPU là lựa chọn có chủ đích cho pilot: đơn giản, rẻ và đủ cho quy mô thử nghiệm. Khi mở rộng, các dịch vụ trạng thái (PostgreSQL, MinIO) và dịch vụ tính toán nặng (vLLM) hoàn toàn có thể tách ra các máy riêng mà không phải đổi kiến trúc, vì chúng đã giao tiếp qua API và biến môi trường ngay từ đầu.

10 Kết Luận và Hướng Phát Triển

10.1 Kết Luận

Báo cáo đã trình bày hành trình từ một bài toán du lịch thực tế đến một hệ thống chạy được, dưới lăng kính của tư duy tính toán. Xuất phát từ ba nút thắt của ngành du lịch — thông tin phân tán, thiếu chiều sâu văn hóa và không phản ứng theo bối cảnh — nhóm đã lần lượt áp dụng đủ bốn trụ cột của Computational Thinking: **decomposition** để chia bài toán thành bốn khối chức năng, **pattern recognition** để nhận ra các mẫu lặp như ra quyết định theo ngữ cảnh và xác minh trước khi cập nhật trạng thái, **abstraction** để biến những khái niệm giàu ngữ nghĩa thành các thực thể và đại lượng tính toán được, và cuối cùng là **algorithm design** để biến chúng thành các quy trình thực thi được.

Điểm nhấn của sản phẩm cuối cùng nằm ở chỗ ba tính năng trọng tâm đều đã được hiện thực và kiểm chứng: bản đồ bong bóng thời gian thực neo vào vị trí GPS thật của người dùng; Mission Path trình bày toàn bộ kho nhiệm vụ văn hóa thành một lộ trình trực quan; và quan trọng nhất, lớp xác minh ảnh đã chuyển từ một bản giả lập chấp nhận mọi ảnh thành một pipeline thị giác thật, chấm điểm theo rubric tường minh trên một mô hình VLM tự host. Chính sự chuyển dịch này thể hiện rõ giá trị của bước evaluation: nếu không kiểm thử hành vi, lỗi “chấp nhận ảnh sai” đã không bị phát hiện.

Xuyên suốt quá trình, một quyết định thiết kế được giữ vững là **đóng khung vai trò của AI**. Thay vì để mô hình hoạt động như một hộp đen tự do, hệ thống luôn cấp cho nó dữ liệu nền (ảnh tham chiếu, mô tả nhiệm vụ), ép nó trả về kết quả có cấu trúc theo rubric, và để các ngưỡng số ở backend đưa ra quyết định cuối. Đây chính là biểu hiện cụ thể nhất của tư duy tính toán trong một hệ thống có AI: biến cái định tính thành cái kiểm soát được.

10.2 Hướng Phát Triển

Từ nền tảng pilot hiện tại, nhóm xác định một số hướng phát triển rõ ràng, tóm tắt trong Bảng 19.

Table 19: Các hướng phát triển tiếp theo

Hướng phát triển	Nội dung
Nhiệm vụ cá nhân hóa	Hiện kho nhiệm vụ dùng chung cho mọi người dùng; bước tiếp theo là chọn lọc và sắp xếp nhiệm vụ theo vị trí, sở thích và lịch sử của từng người.
Củng cố ảnh tham chiếu	Thay vì chỉ lấy ảnh đầu tiên từ tìm kiếm web, có thể tuyển và lưu sẵn một bộ ảnh tham chiếu chất lượng cho mỗi nhiệm vụ để tăng độ chính xác của việc chấm điểm.
Kiểm thử tự động	Bổ sung bộ kiểm thử tự động cho backend và một tập ảnh chuẩn (đúng/sai) để đo độ chính xác của Capture Judge một cách định lượng.
Bổ sung yếu tố ngữ cảnh	Đưa thêm giao thông và mật độ đông đúc vào mô hình bong bóng, tiến gần hơn tới tầm nhìn ban đầu về suitability score đa yếu tố.
Mở rộng ngoài Huế	Nhân rộng kho dữ liệu văn hóa và nhiệm vụ sang các địa phương khác, tận dụng kiến trúc đã tách bạch sẵn giữa nội dung và luồng điều hướng.

Tựu trung, BeVietnam ở giai đoạn pilot đã chứng minh được tính khả thi của một hệ thống du lịch văn hóa thông minh có khả năng phản ứng theo bối cảnh và xác minh được trải nghiệm thực địa. Quan trọng hơn cả sản phẩm, quá trình xây dựng nó là một minh họa hoàn chỉnh cho việc áp dụng tư duy tính toán vào một bài toán thực tế nhiều ràng buộc, từ lúc phân tích vấn đề cho tới khi vận hành một hệ thống có AI được kiểm soát chặt chẽ.

References

- [1] Bộ Văn hóa, Thể thao và Du lịch, “Du lịch việt nam năm 2025: Dấu mốc bất phá, khẳng định vị thế mới,” <https://bvhttdl.gov.vn/du-lich-viet-nam-nam-2025-dau-moc-but-pha-khang-dinh-vi-the-moi-20260108102401774.htm>, 2026, khách quốc tế năm 2025 đạt gần 21,2 triệu lượt, tăng hơn 20% so với 2024. Truy cập 2026-06-23.
- [2] Cục Du lịch Quốc gia Việt Nam, “Du lịch – điểm sáng trong bức tranh kinh tế, xã hội việt nam,” <https://vietnamtourism.gov.vn/post/66500>, 2026, khách nội địa năm 2025 khoảng 135 triệu lượt; tổng thu từ khách du lịch lần đầu vượt 1 triệu tỷ đồng. Truy cập 2026-06-23.
- [3] Bộ Văn hóa, Thể thao và Du lịch, “Những dấu ấn nổi bật của du lịch việt nam năm 2025,” <https://bvhttdl.gov.vn/nhung-dau-an-noi-bat-cua-du-lich-viet-nam-nam-2025-20260105143951138.htm>, 2026, mục tiêu 2026: 25 triệu lượt khách quốc tế, 150 triệu lượt khách nội địa, tổng thu khoảng 1,125 triệu tỷ đồng. Truy cập 2026-06-23.
- [4] Minh Tuấn, “Tìm giải pháp thu hút du khách quay lại việt nam nhiều lần,” <https://hanoimoi.vn/tim-giai-phap-thu-hut-du-khach-quay-lai-viet-nam-nhieu-lan-660544.html>, 2024, báo Hà Nội Mới, 12-03-2024, dẫn số liệu PATA: tỷ lệ khách quốc tế quay lại Thái Lan đạt 80%, Việt Nam khoảng 10–40%. Truy cập 2026-06-23.
- [5] J. M. Wing, “Computational thinking,” *Communications of the ACM*, vol. 49, no. 3, pp. 33–35, 2006.
- [6] P. Spronck, *Computational Thinking with Python*, 2023. [Online]. Available: <https://www.spronck.net/pythonbook/ctbook.pdf>
- [7] S. D. Jesús and D. Martinez, *Applied Computational Thinking with Python: Design Algorithmic Solutions for Complex and Challenging Real-World Problems*. Packt Publishing, 2020.
- [8] BeVietnam Team, “Bevietnam project specification,” Internal project document, 2026.
- [9] OpenStax, “Introduction to computer science: Computational thinking,” <https://openstax.org/books/introduction-computer-science/pages/2-1-computational-thinking>, 2024, accessed 2026-06-21.
- [10] R. W. Sinnott, “Virtues of the haversine,” *Sky and Telescope*, vol. 68, no. 2, p. 159, 1984.
- [11] Foursquare, “Foursquare places API documentation,” <https://docs.foursquare.com>, 2025.
- [12] OpenWeather, “Openweather API documentation,” <https://openweathermap.org/api>, 2025.
- [13] SerpApi, “SerpApi: Google images results API,” <https://serpapi.com>, 2025.
- [14] Qwen Team, “Qwen2.5-VL: Vision-language model series,” <https://github.com/QwenLM/Qwen2.5-VL>, 2025.
- [15] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023.
- [16] vLLM Project, “vLLM: Easy, fast, and cheap LLM serving,” <https://docs.vllm.ai>, 2025.